Spatial Copper Prediction and Handwritten Digit Classification

DASC 3230 · Individual project 2 · Weeks 4–6 · 100 marks

Predict copper concentration at held-out drill holes with Gaussian kernel regression and fully connected neural networks. Then use a fixed small MNIST subset to detect digit 5 and recognize all ten digits.

Complete **25 TODOs**, **eight code-reading responses (S1–S8)** and **six result responses (R1–R6)**. Replace each `raise NotImplementedError` with your implementation. Keep the supplied interfaces, settings and checks. All task instructions, assessment rules and submission requirements are included in this notebook. Read data/README.pdf for the data sources and fixed splits.

For S1–S8 and R1–R6, answer each labelled part separately. Use brief explanations and the specific code, numbers, or plots requested.

Use NumPy, Matplotlib, PyTorch and the Python standard library. The only permitted scikit-learn functions are `roc_curve`, `precision_recall_curve` and `roc_auc_score`, in the supplied plotting/evaluation code. Complete the assessed metrics yourself. Use the supplied local data. **Do not use other modelling libraries, pretrained models or additional data**. Run all models on CPU with one PyTorch thread, as configured below. Keep the supplied splits, candidates and training budgets. Do not refit models on validation or test data.

Do not change any random seed specified in this notebook. Keep all supplied seed values and defaults, including 3230, unchanged throughout the project.

Complete each task's model selection before evaluating its test set. Follow TODO 1–25 in order: complete the regression choices and R1 before the copper test evaluation in TODO 13–14, then complete the MNIST choices and R4 before the MNIST test evaluation in TODO 23–25. Test results must not change models, checkpoints or thresholds. No minimum accuracy or prescribed winning model is required.

Submit only **executed.pdf**, exported from your fully executed notebook. Every page must be **A4 (210 × 297 mm)**. Include all code, execution outputs, figures, and written answers.

Mark allocation

Component	Tasks and written answers	Marks
Spatial data and Gaussian kernel regression	TODO 1–5; S1–S2	25
Shared neural training	TODO 6–9; S3–S4	25
Regression selection and evaluation	TODO 10–14; R1–R3	15
MNIST development	TODO 15–22; S5–S8; R4	20
MNIST final evaluation	TODO 23–25; R5–R6	15

Component	Tasks and written answers	Marks
Total		**100**

An upstream error is assessed at its source. Correct downstream work can receive credit using carried-forward results. Failed checks locate problems; their count is not a deduction count.

In [1]:

```
from pathlib import Path
import json
import copy
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn, optim
from torch.utils.data import DataLoader, TensorDataset

data_candidates = [Path("data"), Path("../individual_project_v2_student/data"),
                   Path("individual_project_v2_student/data")]
DATA_DIR = next((p for p in data_candidates if (p / "mnist_small.npz").exists()), None)
if DATA_DIR is None:
    raise FileNotFoundError("Open this notebook from its student or TA folder, with the
supplied data present.")
OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)
torch.set_num_threads(1)
torch.manual_seed(3230)  # Do not change this seed.
np.set_printoptions(precision=4, suppress=True)
plt.rcParams.update({"figure.figsize": (7, 4), "font.size": 10})
print("PyTorch:", torch.__version__, "| NumPy:", np.__version__)
print("CPU threads:", torch.get_num_threads(), "| Data:", DATA_DIR.resolve())
```

```
PyTorch: 2.14.1 | NumPy: 2.5.4
CPU threads: 1 | Data: /Users/perislyimo/Desktop/DASC
3230/PROJECTS/individual_project_2/data
```

A. Copper grade from spatial coordinates

The supplied GeoMet data contain 2,000 rock samples from 119 drill holes. Each row gives a sample's location and measured copper concentration in ppm. The published data are an obfuscated version of measurements from a real deposit. Use only **x, y, z** to predict **copper_ppm**. The sample and hole identifiers are bookkeeping fields, not predictors.

The supplied split assigns entire drill holes to training, validation, or test. This split evaluates unseen drill holes within the surveyed deposit. All regression models use this same split. A supplied location plot shows the training, validation, and test coordinates. Keep the test copper targets and predictive evaluation reserved until **Final copper evaluation** (TODO 13–14), after the regression model choices and R1 are complete.

TODO 1: define `reg_X` with shape `(2000, 3)` in x, y, z order and `reg_y` with shape `(2000,)` from the loaded structured array.

In [2]:

```
import copy
import json
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn, optim
from torch.utils.data import DataLoader, TensorDataset

REG_SEED = 3230  # Do not change this seed.
torch.set_num_threads(1)
reg_raw = np.genfromtxt(DATA_DIR / "spatial_copper.csv", delimiter=",", names=True)
reg_holes = reg_raw["hole_id"].astype(int)
reg_sample_ids = reg_raw["sample_id"].astype(int)
assert len(reg_raw) == 2000 and len(np.unique(reg_holes)) == 119

# TODO 1
reg_X = np.column_stack([reg_raw["x"], reg_raw["y"], reg_raw["z"]]).astype(float)
reg_y = np.asarray(reg_raw["copper_ppm"], dtype=float)

assert reg_X.shape == (2000, 3) and reg_y.shape == (2000,)
assert np.isfinite(reg_X).all() and np.isfinite(reg_y).all()
assert (reg_y >= 0).all()

with np.load(DATA_DIR / "spatial_split.npz") as saved_split:
    reg_train_idx = saved_split["train_idx"].copy()
    reg_val_idx = saved_split["val_idx"].copy()
    reg_test_idx = saved_split["test_idx"].copy()
assert tuple(map(len, (reg_train_idx, reg_val_idx, reg_test_idx))) == (1164, 412, 424)
assert np.array_equal(np.sort(np.concatenate(
    [reg_train_idx, reg_val_idx, reg_test_idx])), np.arange(len(reg_X)))
for first, second in [(reg_train_idx, reg_val_idx),
                      (reg_train_idx, reg_test_idx), (reg_val_idx, reg_test_idx)]:
    assert np.intersect1d(reg_holes[first], reg_holes[second]).size == 0

reg_X_train, reg_y_train = reg_X[reg_train_idx], reg_y[reg_train_idx]
reg_X_val, reg_y_val = reg_X[reg_val_idx], reg_y[reg_val_idx]
print("Training rows:", len(reg_train_idx), "Validation rows:", len(reg_val_idx),
      "Test rows:", len(reg_test_idx))
print("Training holes:", len(np.unique(reg_holes[reg_train_idx])),
      "Validation holes:", len(np.unique(reg_holes[reg_val_idx])),
      "Test holes:", len(np.unique(reg_holes[reg_test_idx])))
```

```
Training rows: 1164 Validation rows: 412 Test rows: 424
Training holes: 71 Validation holes: 24 Test holes: 24
```

S1 — What does the drill-hole split evaluate? Refer to the data-loading and split-checking code in TODO 1. Answer (a)–(c) separately.

(a) Which of these situations does the supplied split evaluate?

- **Situation A:** Predicting additional samples from a drill hole that already has samples in the training set.
- **Situation B:** Predicting samples from a drill hole with **no samples in the training set**.

Choose A or B, and explain how keeping a whole drill hole in one set gives this result.

(b) Suppose we instead split individual rows at random. Nearby samples from the same hole could then appear in both training and validation. Explain why this could make the validation error too optimistic for predicting a completely unseen drill hole. Focus on the similarity of nearby samples within a hole.

(c) Among the supplied assertions, copy-paste the full `assert` statement that checks that two sets have no drill-hole IDs in common. Explain what its `size == 0` condition means.

Answer:

(a) **Situation B.** Each drill hole is assigned entirely to one of the training, validation or test sets, so every validation or test sample belongs to a hole that has no samples in the training set. The model has never seen any sample from that hole, which is exactly "predicting a drill hole with no training samples".

(b) Samples taken from the same hole lie close together along the core, so their copper grades and coordinates are very similar (spatial autocorrelation). With a random row split, a validation sample would have near-duplicate neighbours in the training set, and the model could do well just by copying those neighbours. That error would look better than the error for a completely new hole, where no close training sample exists, so it would be too optimistic.

(c) `assert np.intersect1d(reg_holes[first], reg_holes[second]).size == 0`

`np.intersect1d` returns the hole IDs that appear in both sets. `size == 0` means that this intersection is empty, i.e. no drill-hole ID is shared between the two sets.

Supplied view of sample locations

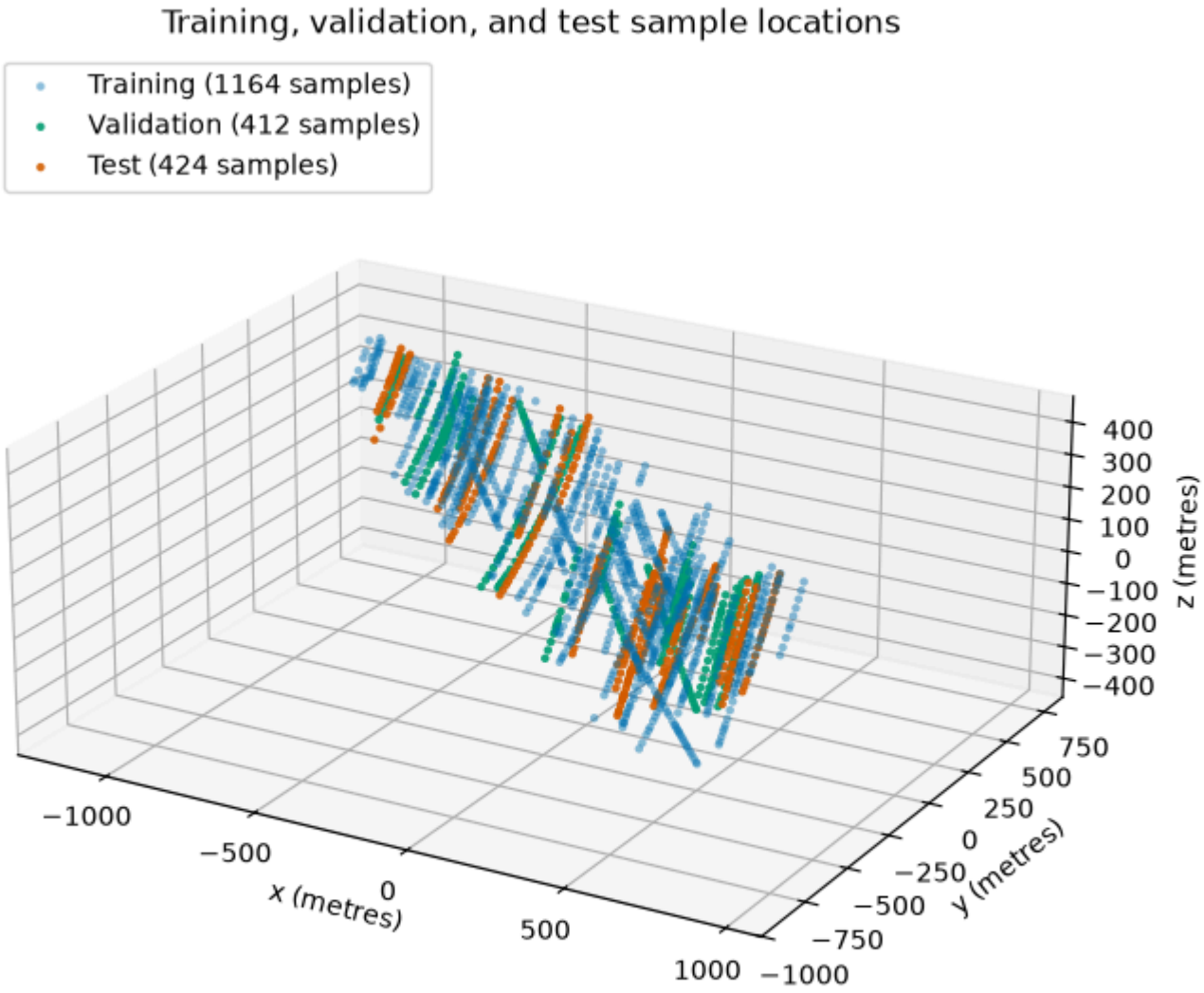
The three-dimensional plot shows **training locations in blue**, **validation locations in green**, and **test locations in orange**, using raw x, y, and z coordinates in metres. Its colors identify the fixed split; they do not represent copper concentration. Use this view to see where the three sets lie, not to change the split or select a model.

This plot does not display test copper targets.

In [3]:

```
# Supplied split view: use coordinates and split membership only.
fig = plt.figure(figsize=(9, 6), constrained_layout=True)
ax = fig.add_subplot(111, projection="3d")
for label, indices, color, opacity in [
        ("Training", reg_train_idx, "#0072B2", 0.45),
        ("Validation", reg_val_idx, "#009E73", 0.9),
        ("Test", reg_test_idx, "#D55E00", 0.9)]:
    reg_location_points = reg_X[indices]
    ax.scatter(reg_location_points[:, 0], reg_location_points[:, 1],
               reg_location_points[:, 2], color=color, s=10, alpha=opacity,
               depthshade=False, edgecolors="none",
               label=f"{label} ({len(indices)} samples)")
reg_location_extent = np.ptp(
    reg_X[np.concatenate([reg_train_idx, reg_val_idx, reg_test_idx])], axis=0)
ax.set_box_aspect(reg_location_extent)
ax.view_init(elev=22, azim=-60)
ax.set(xlabel="x (metres)", ylabel="y (metres)", zlabel="z (metres)",
       title="Training, validation, and test sample locations")
```

```
ax.legend(loc="upper left")
fig.savefig(OUTPUT_DIR / "regression_split_locations_3d.png", dpi=150)
plt.show()
```



Supplied coordinate standardization and baseline

Standardize **each coordinate separately**: subtract that coordinate's training mean and divide by its training standard deviation. Both regression methods use these same standardized inputs.

The neural network predicts a standardized target for numerical stability. Target mean and standard deviation come from training data only. Convert predictions back to ppm before reporting errors. These helpers are supplied.

In [4]:

```
def regression_mse(observed, predicted):
    observed = np.asarray(observed).reshape(-1)
    predicted = np.asarray(predicted).reshape(-1)
    assert observed.shape == predicted.shape
    return float(np.mean((observed - predicted) ** 2))
```

```
reg_x_mean = reg_X_train.mean(axis=0)
reg_x_std = reg_X_train.std(axis=0, ddof=0)
assert reg_x_mean.shape == reg_x_std.shape == (3,)
assert np.all(reg_x_std > 0)
reg_y_mean = float(reg_y_train.mean())
reg_y_scale = float(reg_y_train.std())
assert reg_y_scale > 0

def standardize_reg_coordinates(raw_coordinates):
    return (np.asarray(raw_coordinates, dtype=float) - reg_x_mean) / reg_x_std

reg_Z_train = standardize_reg_coordinates(reg_X_train)
reg_Z_val = standardize_reg_coordinates(reg_X_val)
assert np.allclose(reg_Z_train.mean(axis=0), np.zeros(3), atol=1e-12)
assert np.allclose(reg_Z_train.std(axis=0, ddof=0), np.ones(3), atol=1e-12)
reg_t_train = (reg_y_train - reg_y_mean) / reg_y_scale
reg_t_val = (reg_y_val - reg_y_mean) / reg_y_scale
reg_mean_val_rmse = np.sqrt(regression_mse(
    reg_y_val, np.full(len(reg_y_val), reg_y_mean)))
print(f"Training target mean: {reg_y_mean:.1f} ppm")
print("Training coordinate means (x, y, z; metres):", reg_x_mean)
print("Training coordinate standard deviations (x, y, z; metres):", reg_x_std)
print(f"Mean-predictor validation RMSE: {reg_mean_val_rmse:.1f} ppm")
```

```
Training target mean: 7611.9 ppm
Training coordinate means (x, y, z; metres): [-36.0509  43.7583  84.1493]
Training coordinate standard deviations (x, y, z; metres): [542.107  404.7193 157.9298]
Mean-predictor validation RMSE: 8677.0 ppm
```

B. Gaussian kernel regression

For query q and training locations x_i, both in **standardized coordinates**, compute

$$w_i(q) = \frac{\exp(-\|q - x_i\|^2/(2\sigma^2))}{\sum_j \exp(-\|q - x_j\|^2/(2\sigma^2))}, \qquad \hat{y}(q) = \sum_i w_i(q) y_i.$$

TODO 2: implement `gaussian_weights`. Return an array of shape `(number_of_queries, number_of_training_rows)`. Each row must sum to one. Compute squared Euclidean distances using all coordinate columns.

In [5]:

```
def gaussian_weights(query, centers, sigma):
    query = np.asarray(query, dtype=float)
    centers = np.asarray(centers, dtype=float)
    if query.ndim != 2 or centers.ndim != 2 or query.shape[1] != centers.shape[1]:
        raise ValueError("query and centers must be 2D with the same feature count")
    if sigma <= 0 or len(centers) == 0:
        raise ValueError("sigma must be positive and centers must be nonempty")
    # TODO 2
    squared_distance = ((query[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
    log_weights = -squared_distance / (2.0 * sigma ** 2)
    # Subtracting each row's maximum leaves the normalized weights unchanged
    # but avoids underflow when sigma is small.
    log_weights -= log_weights.max(axis=1, keepdims=True)
    weights = np.exp(log_weights)
    return weights / weights.sum(axis=1, keepdims=True)
```

```
toy_centers = np.array([[0., 0.], [1., 0.]])
toy_w = gaussian_weights(np.array([[0., 0.], [0.5, 0.]]), toy_centers, 1.)
expected_first = np.array([1., np.exp(-0.5)]) / (1. + np.exp(-0.5))
assert np.allclose(toy_w[0], expected_first)
assert np.allclose(toy_w[1], [0.5, 0.5])
assert np.allclose(toy_w.sum(axis=1), 1.)
# Same x coordinate, different y and z: all three coordinate columns matter.
toy_3d_centers = np.array([[0., 0., 0.], [0., 2., 0.], [0., 0., 2.]])
toy_3d_expected = np.array([1., np.exp(-2.), np.exp(-2.)])
toy_3d_expected /= toy_3d_expected.sum()
assert np.allclose(gaussian_weights(np.zeros((1, 3)), toy_3d_centers, 1.)[0],
                   toy_3d_expected)
print("Kernel weight checks passed.")
```

```
Kernel weight checks passed.
```

TODO 3: implement `kernel_predict` using `gaussian_weights` and matrix multiplication. Process at most `query_batch_size` queries at a time. Return one prediction per query, in its original order. Training targets are already in ppm.

In [6]:

```
def kernel_predict(query, centers, targets, sigma, query_batch_size=256):
    query = np.asarray(query, dtype=float)
    targets = np.asarray(targets, dtype=float).reshape(-1)
    if len(centers) != len(targets) or query_batch_size < 1:
        raise ValueError("Check target count and query_batch_size")
    # TODO 3
    predictions = np.empty(len(query), dtype=float)
    for start in range(0, len(query), query_batch_size):
        stop = start + query_batch_size
        weights = gaussian_weights(query[start:stop], centers, sigma)
        predictions[start:stop] = weights @ targets
    return predictions

toy_q = np.array([[0., 0.], [0.5, 0.], [1., 0.]])
toy_pred = kernel_predict(toy_q, toy_centers, np.array([2., 6.]), 1., 2)
expected_left = (2. + 6. * np.exp(-0.5)) / (1. + np.exp(-0.5))
assert toy_pred.shape == (3,)
assert np.allclose(toy_pred, [expected_left, 4., 8. - expected_left])
assert np.allclose(kernel_predict(toy_q, toy_centers, np.full(2, 7.), 0.2, 1), 7.)
print("Kernel prediction checks passed.")
```

```
Kernel prediction checks passed.
```

TODO 4: select a bandwidth using validation MSE. For each bandwidth in `REG_SIGMAS`, predict `reg_Z_val` using `reg_Z_train` as the kernel centers and `reg_y_train` as their targets. Compute MSE against `reg_y_val`, and store the six values in `reg_kernel_val_mse`, with shape `(6,)`, in the supplied bandwidth order. Pass each dimensionless `sigma` directly to `kernel_predict`.

Set `reg_kernel_best_index` to the index with the smallest validation MSE. If two or more bandwidths have exactly the same minimum validation MSE, choose the first one in the supplied order.

In [7]:

```
REG_SIGMAS = np.array([0.1, 0.2, 0.4, 0.8, 1.6, 3.2])
# TODO 4
reg_kernel_val_mse = np.array([
    regression_mse(reg_y_val, kernel_predict(reg_Z_val, reg_Z_train, reg_y_train,
sigma))
    for sigma in REG_SIGMAS])
reg_kernel_best_index = int(np.argmin(reg_kernel_val_mse))  # first index in an exact
tie

assert reg_kernel_val_mse.shape == (6,)
assert np.isfinite(reg_kernel_val_mse).all()
assert 0 <= reg_kernel_best_index < len(REG_SIGMAS)
reg_best_sigma = float(REG_SIGMAS[reg_kernel_best_index])
candidate_summaries = {"regression_kernel": []}
for sigma, val_mse in zip(REG_SIGMAS, reg_kernel_val_mse):
    row = {"sigma": float(sigma), "val_rmse_ppm": float(np.sqrt(val_mse))}
    candidate_summaries["regression_kernel"].append(row)
    print(f"sigma={sigma:3.1f} (dimensionless)  validation MSE={val_mse:12.1f} ppm²  "
          f"validation RMSE={np.sqrt(val_mse):8.1f} ppm")
print(f"Selected bandwidth: {reg_best_sigma:g} (dimensionless)")
```

```
sigma=0.1 (dimensionless)  validation MSE=  71299330.2 ppm²  validation RMSE=  8443.9
ppm
sigma=0.2 (dimensionless)  validation MSE=  69295751.3 ppm²  validation RMSE=  8324.4
ppm
sigma=0.4 (dimensionless)  validation MSE=  71317641.2 ppm²  validation RMSE=  8445.0
ppm
sigma=0.8 (dimensionless)  validation MSE=  73667358.0 ppm²  validation RMSE=  8583.0
ppm
sigma=1.6 (dimensionless)  validation MSE=  74744033.1 ppm²  validation RMSE=  8645.5
ppm
sigma=3.2 (dimensionless)  validation MSE=  75165870.0 ppm²  validation RMSE=  8669.8
ppm
Selected bandwidth: 0.2 (dimensionless)
```

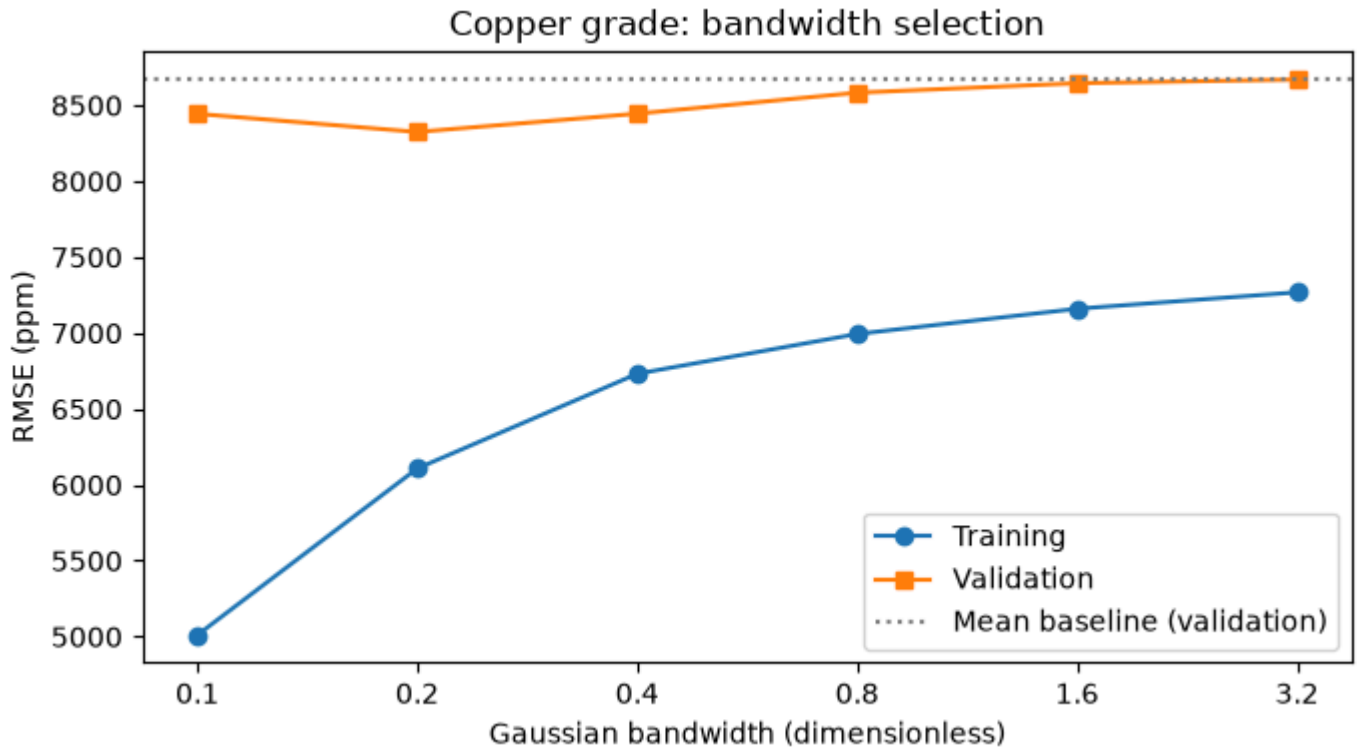
TODO 5: compute training MSE for the six bandwidths in `REG_SIGMAS`, and store the values in `reg_kernel_train_mse`, with shape `(6,)`, in the supplied order. Predict `reg_Z_train` using all of `reg_Z_train` as the kernel centers and `reg_y_train` as their targets, then compute MSE against `reg_y_train`. Each training sample must remain among the kernel centers when predicting itself.

Convert both MSE arrays to RMSE by taking square roots; reuse `reg_kernel_val_mse` from TODO 4. Plot both RMSE curves against the dimensionless bandwidths on the supplied axes. Label the curves `Training` and `Validation` for the supplied legend. These curves show how training and validation errors change with bandwidth; keep the bandwidth selected using validation MSE in TODO 4.

In [8]:

```
fig, ax = plt.subplots(figsize=(7, 4))
# TODO 5
reg_kernel_train_mse = np.array([
    regression_mse(reg_y_train, kernel_predict(reg_Z_train, reg_Z_train, reg_y_train,
sigma))
    for sigma in REG_SIGMAS])
ax.plot(REG_SIGMAS, np.sqrt(reg_kernel_train_mse), marker="o", label="Training")
ax.plot(REG_SIGMAS, np.sqrt(reg_kernel_val_mse), marker="s", label="Validation")
```

```
assert reg_kernel_train_mse.shape == (6,)
assert np.isfinite(reg_kernel_train_mse).all()
for row, train_mse in zip(candidate_summaries["regression_kernel"],
reg_kernel_train_mse):
    row["train_rmse_ppm"] = float(np.sqrt(train_mse))
ax.axhline(reg_mean_val_rmse, color="gray", linestyle=":", label="Mean baseline
(validation)")
ax.set_xscale("log", base=2)
ax.set_xticks(REG_SIGMAS, [f"{s:g}" for s in REG_SIGMAS])
ax.set(xlabel="Gaussian bandwidth (dimensionless)", ylabel="RMSE (ppm)",
       title="Copper grade: bandwidth selection")
ax.legend()
fig.tight_layout()
plt.show()
```



Inspect one validation prediction

The supplied code selects the first validation query and prints its five largest kernel weights. The final prediction still uses **all** training samples. The displayed `distance_std` is Euclidean distance in standardized coordinates, so it is dimensionless rather than a distance in meters.

In [9]:

```
reg_query_index = 0
reg_query_weights = gaussian_weights(reg_Z_val[[reg_query_index]], reg_Z_train,
                                     reg_best_sigma)[0]
reg_top5 = np.argsort(-reg_query_weights, kind="stable")[:5]
reg_query_prediction = float(reg_query_weights @ reg_y_train)
print("Query sample ID:", reg_sample_ids[reg_val_idx[reg_query_index]])
print(" sample_id  distance_std   weight   copper_ppm   contribution_ppm")
for index in reg_top5:
    distance_std = np.linalg.norm(reg_Z_train[index] - reg_Z_val[reg_query_index])
    print(f"{reg_sample_ids[reg_train_idx[index]]:10d} {distance_std:13.4f} "
          f"{reg_query_weights[index]:8.4f} {reg_y_train[index]:12.1f} "
```

```
          f"{reg_query_weights[index] * reg_y_train[index]:18.2f}")
print(f"Top-five total weight: {reg_query_weights[reg_top5].sum():.4f}")
print(f"Full prediction: {reg_query_prediction:.1f} ppm; "
      f"observed: {reg_y_val[reg_query_index]:.1f} ppm")
```

```
Query sample ID: 71
 sample_id  distance_std   weight   copper_ppm   contribution_ppm
       706        0.1042   0.1666       3600.0             599.90
       707        0.1313   0.1539       5600.0             861.70
       639        0.1909   0.1211      15000.0            1815.77
       708        0.2255   0.1011       6100.0             616.53
       640        0.2389   0.0935      10700.0            1000.95
Top-five total weight: 0.6362
Full prediction: 6600.0 ppm; observed: 4100.0 ppm
```

S2 — Read one kernel prediction. Use the output under **Inspect one validation prediction**. Keep the displayed query and selected bandwidth fixed. In this question, closer/farther refers to `distance_std` in standardized coordinates.

(a) Which receives the larger Gaussian weight: a closer or a more distant training sample? Explain using the distance term in the kernel formula.

(b) Are the weights normalized over these five samples or over all training samples?

(c) Using one printed row, explain how `weight` and `copper_ppm` produce `contribution_ppm`.

Answer:

(a) The **closer** training sample gets the larger weight. The weight is proportional to exp(-||q - x_i||² / (2σ²)). A larger distance gives a larger squared distance and a more negative exponent, so a smaller exponential. In the output, distance_std 0.1042 gets weight 0.1666 while distance_std 0.2389 gets only 0.0935.

(b) The weights are normalized over **all training samples**, not over these five. The denominator of the formula is the sum over every training sample j. The five printed weights add up to only 0.6362 (less than 1); the remaining training samples carry the other 0.3638 of the weight.

(c) For sample_id 706: contribution_ppm = weight × copper_ppm = 0.1666 × 3600.0 ≈ 599.9 ppm (the small difference is rounding of the printed weight). The full prediction is the sum of these contributions over all training samples, which gives the printed 6600.0 ppm.

C. Neural network regression

Use three fixed architectures: `3 → 16 → 1`, `3 → 32 → 1`, and `3 → 32 → 32 → 1`. Place ReLU after every hidden layer. The final layer returns one real number and has no activation.

TODO 6: complete `build_regressor(hidden_sizes)` with `nn.Linear`, `nn.ReLU`, and `nn.Sequential`. The list `hidden_sizes` contains one or two hidden-layer widths: for example, `[16]` means one hidden layer with 16 units, and `[32, 32]` means two hidden layers with 32 units each.

Handle the one-hidden-layer and two-hidden-layer cases separately with `if` and `elif`. In each case, list the layers directly inside `nn.Sequential(...)`, as in Week 5. Read the widths from `hidden_sizes` so the function also works with the smaller widths used in the supplied check. Keep that check.

In [10]:

```
def build_regressor(hidden_sizes):
    # TODO 6
    if len(hidden_sizes) == 1:
        return nn.Sequential(
            nn.Linear(3, hidden_sizes[0]),
            nn.ReLU(),
            nn.Linear(hidden_sizes[0], 1))
    elif len(hidden_sizes) == 2:
        return nn.Sequential(
            nn.Linear(3, hidden_sizes[0]),
            nn.ReLU(),
            nn.Linear(hidden_sizes[0], hidden_sizes[1]),
            nn.ReLU(),
            nn.Linear(hidden_sizes[1], 1))
    else:
        raise ValueError("hidden_sizes must contain one or two widths")

toy_network = build_regressor([4, 2])
assert toy_network(torch.zeros(5, 3)).shape == (5, 1)
assert sum(parameter.numel() for parameter in toy_network.parameters()) == 29
assert sum(isinstance(layer, nn.ReLU) for layer in toy_network.modules()) == 2
assert isinstance(list(toy_network.children())[-1], nn.Linear)
print("Regression architecture checks passed.")
```

```
Regression architecture checks passed.
```

Shared training functions

All supplied loss functions return a **mean over the current batch**.

TODO 7: implement one training epoch. Enter training mode. For each batch, clear gradients, compute loss, run backpropagation, and update parameters. Return the sample-weighted mean of the batch losses recorded during that epoch.

TODO 8: implement loss evaluation with `model.eval()` and `torch.no_grad()`. Return the loss summed over samples, divided by the number of samples. Do not update parameters. The final batch may contain fewer samples than the other batches.

In [11]:

```
def train_one_epoch(model, loader, criterion, optimizer):
    # TODO 7
    model.train()
    total_loss, total_samples = 0.0, 0
    for inputs, targets in loader:
        optimizer.zero_grad()
        loss = criterion(model(inputs), targets)
        loss.backward()
        optimizer.step()
```

```
        total_loss += loss.item() * len(inputs)
        total_samples += len(inputs)
    return total_loss / total_samples

def evaluate_loss(model, loader, criterion):
    # TODO 8
    model.eval()
    total_loss, total_samples = 0.0, 0
    with torch.no_grad():
        for inputs, targets in loader:
            loss = criterion(model(inputs), targets)
            total_loss += loss.item() * len(inputs)
            total_samples += len(inputs)
    return total_loss / total_samples

# An independently known gradient: d[(w - 2)^2]/dw at w=0 is -4.
toy_step_model = nn.Linear(1, 1, bias=False)
with torch.no_grad():
    toy_step_model.weight.zero_()
toy_step_loader = DataLoader(TensorDataset(torch.ones(1, 1), torch.tensor([[2.]])),
                             batch_size=1, num_workers=0)
toy_optimizer = optim.SGD(toy_step_model.parameters(), lr=0.1)
toy_step_loss = train_one_epoch(toy_step_model, toy_step_loader, nn.MSELoss(),
toy_optimizer)
assert np.isclose(toy_step_loss, 4.)
assert torch.allclose(toy_step_model.weight, torch.tensor([[0.4]]))

# Unequal batch sizes: sample losses are 0, 0, 9, so their mean is 3.
toy_loss_model = nn.Linear(1, 1, bias=False)
with torch.no_grad():
    toy_loss_model.weight.zero_()
toy_loss_loader = DataLoader(TensorDataset(torch.ones(3, 1),
    torch.tensor([[0.], [0.], [3.]])), batch_size=2, shuffle=False, num_workers=0)
toy_before = toy_loss_model.weight.detach().clone()
assert np.isclose(evaluate_loss(toy_loss_model, toy_loss_loader, nn.MSELoss()), 3.)
assert torch.equal(toy_before, toy_loss_model.weight.detach())
assert not toy_loss_model.training
print("Gradient-update and weighted-loss checks passed.")
```

```
Gradient-update and weighted-loss checks passed.
```

S3 — Gradients and averaging. Refer to TODO 7–8 and their toy checks.

(a) Suppose `optimizer.zero_grad()` is removed, with the rest of the training loop unchanged. After the second call to `loss.backward()`, would the stored gradient contain only the second batch's contribution, or accumulated contributions from both batches? Explain how this would affect the second parameter update.

(b) In the evaluation toy check, the two batches contain 2 and 1 samples, and their mean losses are 0 and 9. Write the calculation for the mean loss over all 3 samples. Explain why simply averaging the two batch means gives the wrong result.

Answer:

(a) The gradient would **accumulate contributions from both batches**. PyTorch adds new gradients into the stored `.grad` instead of replacing them, so after the second `loss.backward()` the stored

gradient is (gradient of batch 1) + (gradient of batch 2). The second parameter update would then use this sum, so batch 1 is counted twice and the step is the wrong size and direction for batch 2 alone.

(b) Mean loss over all 3 samples = (2 × 0 + 1 × 9) / 3 = 3. Averaging the two batch means gives (0 + 9) / 2 = 4.5, which is wrong because it gives the single-sample batch the same weight as the two-sample batch. Each batch mean must be multiplied by its batch size (or the per-sample losses summed) before dividing by the total number of samples.

TODO 9: implement `fit_network` with the following fixed interface.

- Create an SGD optimizer with `lr`. If `step_size` is supplied, create a StepLR scheduler with that step interval and `gamma`.
- Train for `epochs`. Record the learning rate used for each epoch.
- At each epoch's end, evaluate training and validation loss with the same fixed weights. Store both losses in `history`.
- Keep a deep copy of the weights whenever validation loss is strictly lower. Store the best epoch as a **one-based** integer; retain the earlier epoch in a tie.
- Call the scheduler once at the end of each epoch. Restore the best weights before returning `history`.

Return a dictionary with exactly these fields: `train_loss`, `val_loss`, and `lr` are lists of length `epochs`; `best_epoch` is a one-based integer.

Do not choose a checkpoint with test data. The same function must also work when `step_size=None`, as required for MNIST.

In [12]:

```
def fit_network(model, train_loader, val_loader, criterion, epochs, lr,
                step_size=None, gamma=0.5):
    # TODO 9
    optimizer = optim.SGD(model.parameters(), lr=lr)
    scheduler = (optim.lr_scheduler.StepLR(optimizer, step_size=step_size, gamma=gamma)
                 if step_size is not None else None)
    history = {"train_loss": [], "val_loss": [], "lr": [], "best_epoch": 0}
    best_val_loss, best_state = float("inf"), None
    for epoch in range(1, epochs + 1):
        history["lr"].append(float(optimizer.param_groups[0]["lr"]))
        train_one_epoch(model, train_loader, criterion, optimizer)
        # Losses at the end of the epoch, with the weights held fixed.
        train_loss = evaluate_loss(model, train_loader, criterion)
        val_loss = evaluate_loss(model, val_loader, criterion)
        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        if val_loss < best_val_loss:  # strictly lower: an earlier epoch wins a tie
            best_val_loss = val_loss
            best_state = copy.deepcopy(model.state_dict())
            history["best_epoch"] = epoch
        if scheduler is not None:
            scheduler.step()
    if best_state is not None:
        model.load_state_dict(best_state)
    return history
```

```
# A deliberately overshooting toy fit: epoch 1 is the best validation checkpoint.
toy_fit_model = nn.Linear(1, 1, bias=False)
with torch.no_grad():
    toy_fit_model.weight.zero_()
toy_fit_train = DataLoader(TensorDataset(torch.ones(1, 1), torch.ones(1, 1)),
                           batch_size=1, num_workers=0)
toy_fit_val = DataLoader(TensorDataset(torch.ones(1, 1), torch.tensor([[2.2]])),
                         batch_size=1, num_workers=0)
toy_history = fit_network(toy_fit_model, toy_fit_train, toy_fit_val, nn.MSELoss(),
                          epochs=3, lr=1.1)
assert toy_history["best_epoch"] == 1
assert torch.allclose(toy_fit_model.weight, torch.tensor([[2.2]]), atol=1e-6)
assert np.allclose(toy_history["lr"], [1.1, 1.1, 1.1])
assert len(toy_history["train_loss"]) == len(toy_history["val_loss"]) == 3

toy_schedule_model = nn.Linear(1, 1, bias=False)
with torch.no_grad():
    toy_schedule_model.weight.zero_()
toy_schedule_history = fit_network(toy_schedule_model, toy_fit_train, toy_fit_train,
    nn.MSELoss(), epochs=3, lr=0.1, step_size=2, gamma=0.5)
assert np.allclose(toy_schedule_history["lr"], [0.1, 0.1, 0.05])
print("Best-checkpoint and scheduler checks passed.")
```

```
Best-checkpoint and scheduler checks passed.
```

S4 — Checkpoints and schedules. Refer to `fit_network` in TODO 9.

(a) After saving `best_state = model.state_dict()`, training continues and changes the model parameters. Would this assignment alone preserve the parameter values from the selected epoch? Explain what `copy.deepcopy(...)` changes.

(b) Suppose epoch 80 has the lowest validation loss, but training finishes at epoch 300 with a higher validation loss. Which epoch's weights should be restored into the model before the function returns, and why?

(c) With `StepLR(step_size=100, gamma=0.5)`, when does the first learning-rate reduction occur if `scheduler.step()` is called after every batch? When does it occur if called after every epoch? State the unit in each answer: batches or epochs.

Answer:

(a) No. `model.state_dict()` returns a dictionary that refers to the model's live parameter tensors, not copies. When training continues, the parameters are updated in place, so `best_state` would show the newest values instead of the epoch-80 values. `copy.deepcopy(...)` makes independent copies of the tensors, so the saved values stay fixed at the selected epoch.

(b) The weights from **epoch 80** should be restored. That epoch has the lowest validation loss, so it is the best estimate of performance on unseen data. By epoch 300 validation loss is higher, i.e. the model generalizes worse, even if its training loss is lower. The final epoch is therefore not selected.

(c) If `scheduler.step()` is called after every **batch**, the first reduction occurs after 100 **batches**. If it is called after every **epoch**, the first reduction occurs after 100 **epochs**.

Train and select the regression networks

TODO 10: train each architecture in the supplied order. Before each model, set `torch.manual_seed(REG_SEED)`. Create a new model and new data loaders, then call `fit_network`. Append the trained model and its history to the two lists.

Use **SGD, learning rate 0.03, 300 epochs, batch size 128**, and **StepLR every 100 epochs with gamma 0.5**. Each candidate starts from fresh parameters. The supplied loaders use CPU float32 tensors and a seeded shuffling generator.

In [13]:

```
REG_ARCHITECTURES = [[16], [32], [32, 32]]
REG_EPOCHS, REG_LR, REG_BATCH_SIZE = 300, 0.03, 128

def make_regression_loaders():
    train_dataset = TensorDataset(torch.tensor(reg_Z_train, dtype=torch.float32),
        torch.tensor(reg_t_train[:, None], dtype=torch.float32))
    val_dataset = TensorDataset(torch.tensor(reg_Z_val, dtype=torch.float32),
        torch.tensor(reg_t_val[:, None], dtype=torch.float32))
    generator = torch.Generator().manual_seed(REG_SEED)  # Do not change REG_SEED.
    train_loader = DataLoader(train_dataset, batch_size=REG_BATCH_SIZE, shuffle=True,
                              num_workers=0, generator=generator)
    val_loader = DataLoader(val_dataset, batch_size=REG_BATCH_SIZE, shuffle=False,
                            num_workers=0)
    return train_loader, val_loader

reg_nn_models, reg_nn_histories = [], []
# TODO 10
for widths in REG_ARCHITECTURES:
    torch.manual_seed(REG_SEED)
    model = build_regressor(widths)
    train_loader, val_loader = make_regression_loaders()
    history = fit_network(model, train_loader, val_loader, nn.MSELoss(),
                          epochs=REG_EPOCHS, lr=REG_LR, step_size=100, gamma=0.5)
    reg_nn_models.append(model)
    reg_nn_histories.append(history)

assert len(reg_nn_models) == len(reg_nn_histories) == len(REG_ARCHITECTURES)
for widths, history in zip(REG_ARCHITECTURES, reg_nn_histories):
    assert len(history["val_loss"]) == REG_EPOCHS
    best = history["best_epoch"] - 1
    print(f"Hidden widths {widths}: best epoch={best + 1}, "
          f"validation RMSE={np.sqrt(history['val_loss'][best]) * reg_y_scale:.1f}
ppm")
```

```
Hidden widths [16]: best epoch=279, validation RMSE=8502.8 ppm
Hidden widths [32]: best epoch=279, validation RMSE=8449.4 ppm
Hidden widths [32, 32]: best epoch=279, validation RMSE=8422.7 ppm
```

TODO 11: define `reg_nn_val_mse` as a one-dimensional array containing each network's best validation MSE **in ppm²**. Select `reg_nn_best_index` using the lowest validation MSE; choose the first architecture in the supplied order in an exact tie. The standardized-target MSE must be multiplied by `reg_y_scale ** 2`.

In [14]:

```
# TODO 11
reg_nn_val_mse = np.array([
    history["val_loss"][history["best_epoch"] - 1] * reg_y_scale ** 2
    for history in reg_nn_histories])
reg_nn_best_index = int(np.argmin(reg_nn_val_mse))  # first index in an exact tie

assert reg_nn_val_mse.shape == (len(REG_ARCHITECTURES),)
assert np.isfinite(reg_nn_val_mse).all()
reg_nn_model = reg_nn_models[reg_nn_best_index]
reg_nn_history = reg_nn_histories[reg_nn_best_index]
reg_nn_widths = REG_ARCHITECTURES[reg_nn_best_index].copy()
candidate_summaries["regression_nn"] = []
for widths, history, val_mse in zip(REG_ARCHITECTURES, reg_nn_histories,
reg_nn_val_mse):
    best = history["best_epoch"] - 1
    candidate_summaries["regression_nn"].append({
        "hidden_sizes": widths.copy(), "best_epoch": int(best + 1),
        "val_rmse_ppm": float(np.sqrt(val_mse)),
        "train_rmse_ppm": float(np.sqrt(history["train_loss"][best]) * reg_y_scale)})
print("Selected hidden widths:", reg_nn_widths)
```

```
Selected hidden widths: [32, 32]
```

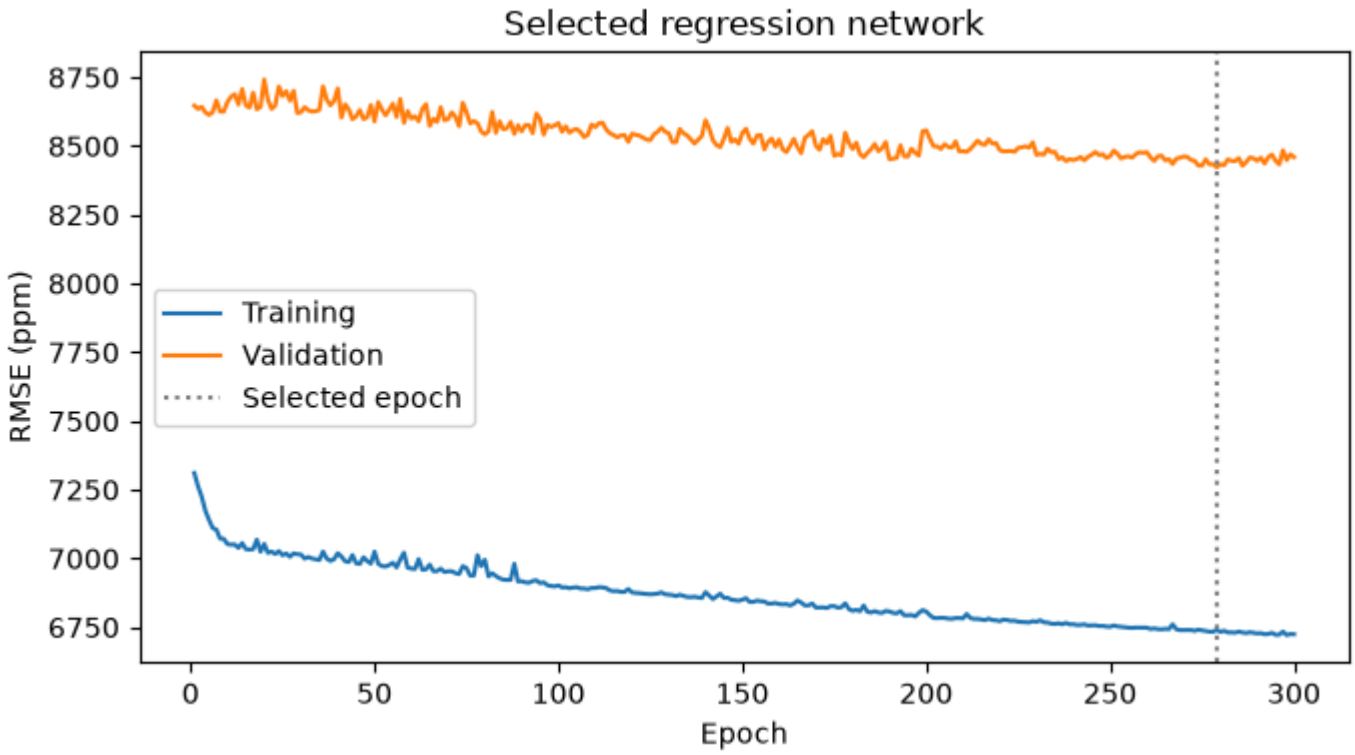
TODO 12: plot the selected network's training and validation RMSE in ppm against epoch. Use the stored losses, evaluated at each epoch's end. The supplied vertical line marks the selected checkpoint.

In [15]:

```
fig, ax = plt.subplots(figsize=(7, 4))
reg_epoch_axis = np.arange(1, REG_EPOCHS + 1)
# TODO 12
reg_nn_train_rmse = np.sqrt(reg_nn_history["train_loss"]) * reg_y_scale
reg_nn_val_rmse = np.sqrt(reg_nn_history["val_loss"]) * reg_y_scale
ax.plot(reg_epoch_axis, reg_nn_train_rmse, label="Training")
ax.plot(reg_epoch_axis, reg_nn_val_rmse, label="Validation")

ax.axvline(reg_nn_history["best_epoch"], color="gray", linestyle=":", label="Selected
epoch")
ax.set(xlabel="Epoch", ylabel="RMSE (ppm)", title="Selected regression network")
ax.legend()
fig.tight_layout()
plt.show()

# Freeze both regression model choices before the copper test evaluation.
reg_frozen = {"kernel_sigma": reg_best_sigma, "kernel_bandwidth_unit": "dimensionless",
              "coordinate_scaling": "per_coordinate_training_zscore",
"coordinate_std_ddof": 0,
              "nn_hidden_sizes": reg_nn_widths,
              "nn_best_epoch": int(reg_nn_history["best_epoch"]),
              "nn_seed": REG_SEED, "nn_initial_lr": REG_LR,
              "nn_epochs": REG_EPOCHS, "nn_batch_size": REG_BATCH_SIZE,
              "nn_step_size": 100, "nn_gamma": 0.5}
print(json.dumps(reg_frozen, indent=2))
```




```
{
  "kernel_sigma": 0.2,
  "kernel_bandwidth_unit": "dimensionless",
  "coordinate_scaling": "per_coordinate_training_zscore",
  "coordinate_std_ddof": 0,
  "nn_hidden_sizes": [
    32,
    32
  ],
  "nn_best_epoch": 279,
  "nn_seed": 3230,
  "nn_initial_lr": 0.03,
  "nn_epochs": 300,
  "nn_batch_size": 128,
  "nn_step_size": 100,
  "nn_gamma": 0.5
}
```


R1 — Record the regression choices before testing. Use the printed results from TODO 4 and TODO 10–12, the mean-baseline output, and the RMSE plots in TODO 5 and TODO 12.

(a) Complete the three labelled lines under **Answer (a)** below. Replace each `[fill in]` with the requested value from your outputs. For the neural network's architecture, give the layer sizes from input to output.

(b) Compare the bandwidth with the smallest training RMSE and the bandwidth with the smallest validation RMSE. Use the MSE arrays from TODO 4–5, taking square roots to obtain RMSE. For each of these two bandwidths, report its value and both its training and validation RMSE in ppm. State whether the two bandwidths are the same or different; if they are the same, report that bandwidth once. If several bandwidths share exactly the same minimum, choose the first in the supplied order.

Using the TODO 5 plot, describe how both RMSE curves change from the smallest to the largest bandwidth. Explain why the final bandwidth is chosen using validation error, even if another bandwidth

has lower training error. Retain the TODO 4 choice.

(c) Using the selected network's learning curves, state whether its selected checkpoint is the last epoch or an earlier epoch, and explain the choice using validation RMSE. If the minimum is tied across epochs, apply the prescribed earliest-epoch rule. Use the observed curves; do not assume that validation error must rise late in training.

Complete this response before testing and retain these choices afterwards.

Answer:

(a)

- Mean baseline: validation RMSE = 8677.0 ppm.
- Selected kernel: validation RMSE = 8324.4 ppm; bandwidth = 0.2 (dimensionless).
- Selected neural network: validation RMSE = 8422.7 ppm; architecture = 3 → 32 → 32 → 1; selected epoch = 279.

(b) The bandwidth with the **smallest training RMSE** is σ = 0.1 (training RMSE 5010.2 ppm, validation RMSE 8443.9 ppm). The bandwidth with the **smallest validation RMSE** is σ = 0.2 (training RMSE 6107.5 ppm, validation RMSE 8324.4 ppm). The two bandwidths are **different**.

From σ = 0.1 to σ = 3.2 the training RMSE rises steadily, from 5010.2 to 7268.3 ppm. The validation RMSE first falls, from 8443.9 ppm at σ = 0.1 to its minimum of 8324.4 ppm at σ = 0.2, and then rises to 8669.8 ppm at σ = 3.2, almost the mean-baseline value of 8677.0 ppm. With a very small bandwidth each training sample is among its own kernel centers and dominates its own prediction, so training error is small but this does not carry over to unseen holes. With a very large bandwidth the prediction averages almost all samples and tends toward the global mean. The bandwidth is chosen with validation error because the validation holes are unseen, as the test holes will be, whereas training error rewards memorization (σ = 0.1 would win on training error but has a worse validation RMSE, 8443.9 versus 8324.4 ppm).

(c) The selected checkpoint is an **earlier epoch** (279 out of 300), not the last one. The validation RMSE is lowest at epoch 279 (about 8422.7 ppm) and is slightly higher by epoch 300, so epoch 279 is the minimum. The validation curve is noisy and drifts downward slowly over most of training, while the training RMSE keeps falling slightly. The checkpoint is chosen by the lowest validation RMSE, not by the training curve. The differences between late epochs are small.

Save and reload the regression predictors

Run the supplied export and reload checks on validation inputs. Kernel regression needs its training locations and targets at prediction time. The neural network needs its architecture, learned weights, and training scaling statistics. Both exports must retain all three coordinate means and standard deviations, so new raw coordinates receive the same transformation used during training.

In [16]:

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
reg_kernel_state = {"x_mean": reg_x_mean.copy(), "x_std": reg_x_std.copy(),
                    "centers": reg_Z_train.copy(), "targets_ppm": reg_y_train.copy(),
```

```
                    "sigma": np.array(reg_best_sigma)}
np.savez(OUTPUT_DIR / "regression_kernel.npz", **reg_kernel_state)
reg_nn_saved = {"hidden_sizes": reg_nn_widths.copy(),
                "state_dict": copy.deepcopy(reg_nn_model.state_dict()),
                "x_mean": reg_x_mean.tolist(), "x_std": reg_x_std.tolist(),
                "y_mean": reg_y_mean, "y_scale": reg_y_scale}
torch.save(reg_nn_saved, OUTPUT_DIR / "regression_nn.pt")
(OUTPUT_DIR / "regression_metadata.json").write_text(
    json.dumps({"selection": reg_frozen,
                "candidate_summaries": {key: value for key, value in
candidate_summaries.items()
                                        if key.startswith("regression_")}}, indent=2))

def predict_kernel_raw(raw_coordinates, state):
    standardized = (np.asarray(raw_coordinates, dtype=float) - state["x_mean"]) /
state["x_std"]
    return kernel_predict(standardized, state["centers"], state["targets_ppm"],
                          float(state["sigma"]))

def predict_regression_nn_raw(raw_coordinates, model, state):
    scaled = ((np.asarray(raw_coordinates, dtype=float) - np.asarray(state["x_mean"]))
              / np.asarray(state["x_std"]))
    model.eval()
    with torch.no_grad():
        standardized = model(torch.tensor(scaled,
dtype=torch.float32)).numpy().reshape(-1)
    return standardized * state["y_scale"] + state["y_mean"]

with np.load(OUTPUT_DIR / "regression_kernel.npz") as saved:
    reg_kernel_reloaded = {key: saved[key].copy() for key in saved.files}
assert reg_kernel_reloaded["x_mean"].shape == reg_kernel_reloaded["x_std"].shape ==
(3,)
reg_nn_reloaded_state = torch.load(OUTPUT_DIR / "regression_nn.pt", map_location="cpu",
                                   weights_only=True)
reg_nn_reloaded_model = build_regressor(reg_nn_reloaded_state["hidden_sizes"])
reg_nn_reloaded_model.load_state_dict(reg_nn_reloaded_state["state_dict"])
assert np.allclose(predict_kernel_raw(reg_X_val[:10], reg_kernel_state),
                   predict_kernel_raw(reg_X_val[:10], reg_kernel_reloaded))
assert np.allclose(predict_regression_nn_raw(reg_X_val[:10], reg_nn_model,
reg_nn_saved),
                   predict_regression_nn_raw(reg_X_val[:10], reg_nn_reloaded_model,
                                             reg_nn_reloaded_state))
print("Regression export and reload checks passed.")
```

```
Regression export and reload checks passed.
```

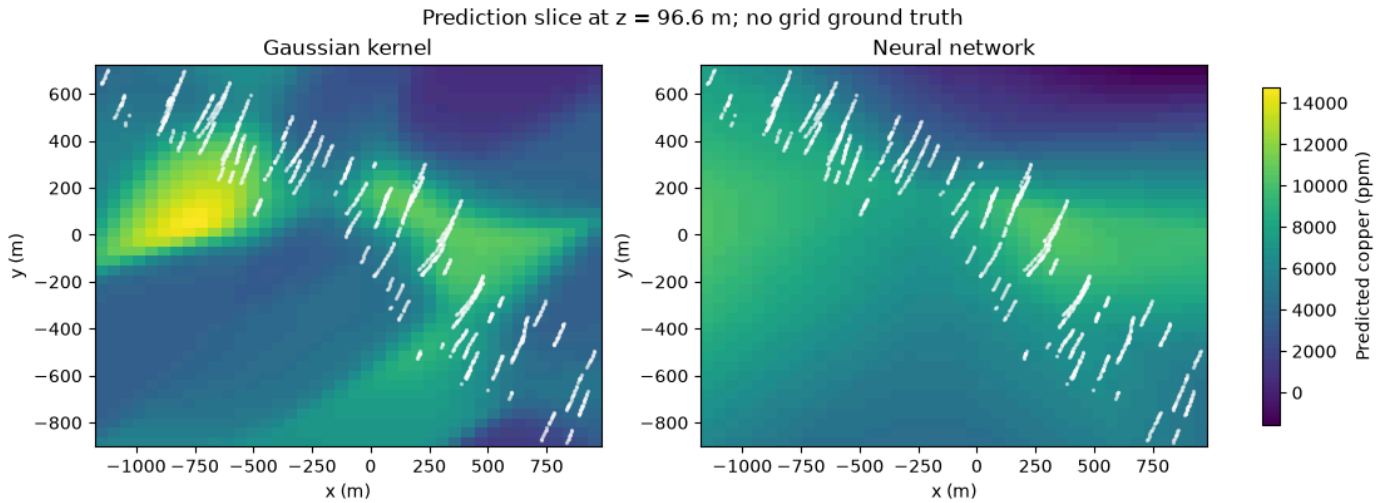
Supplied spatial prediction slice

Compare the two frozen predictors on one common 40 by 40 x/y grid. Its z coordinate is the median training depth, and its x/y range comes from the training samples. Both panels use the same color scale. White dots are **x/y projections of all training samples**, which may lie at other depths.

There are **no measured copper values on this grid**. Use the maps to describe model predictions and differences, not to measure grid accuracy. This is a supplied display, not an additional model-selection step.

In [17]:

```
reg_slice_z = float(np.median(reg_X_train[:, 2]))
reg_slice_x, reg_slice_y = np.meshgrid(
    np.linspace(reg_X_train[:, 0].min(), reg_X_train[:, 0].max(), 40),
    np.linspace(reg_X_train[:, 1].min(), reg_X_train[:, 1].max(), 40))
reg_slice_coordinates = np.column_stack([reg_slice_x.ravel(), reg_slice_y.ravel(),
    np.full(reg_slice_x.size, reg_slice_z)])
reg_slice_kernel = predict_kernel_raw(reg_slice_coordinates,
reg_kernel_reloaded).reshape(40, 40)
reg_slice_nn = predict_regression_nn_raw(reg_slice_coordinates, reg_nn_reloaded_model,
    reg_nn_reloaded_state).reshape(40, 40)
reg_slice_min = min(reg_slice_kernel.min(), reg_slice_nn.min())
reg_slice_max = max(reg_slice_kernel.max(), reg_slice_nn.max())
fig, axes = plt.subplots(1, 2, figsize=(11, 4), constrained_layout=True)
for ax, values, title in zip(axes, [reg_slice_kernel, reg_slice_nn],
                             ["Gaussian kernel", "Neural network"]):
    reg_slice_image = ax.pcolormesh(reg_slice_x, reg_slice_y, values, shading="auto",
        cmap="viridis", vmin=reg_slice_min, vmax=reg_slice_max)
    ax.scatter(reg_X_train[:, 0], reg_X_train[:, 1], color="white", s=2, alpha=0.5)
    ax.set(xlabel="x (m)", ylabel="y (m)", title=title)
    ax.set_aspect("equal", adjustable="box")
fig.colorbar(reg_slice_image, ax=axes, label="Predicted copper (ppm)", shrink=0.85)
fig.suptitle(f"Prediction slice at z = {reg_slice_z:.1f} m; no grid ground truth")
fig.savefig(OUTPUT_DIR / "regression_prediction_slice.png", dpi=150)
plt.show()
```



Final copper evaluation

Complete R1 and fix the regression bandwidth, architecture, and checkpoint before running this section. Use the saved, validation-selected regression models without refitting, and keep these choices fixed after viewing test results. Complete TODO 13–14 and R2–R3 before continuing to MNIST.

TODO 13: define `reg_kernel_test_pred` and `reg_nn_test_pred` with the supplied raw-input prediction functions and reloaded states. Define `reg_test_mse` as a dict with keys `Mean`, `Kernel`, `Neural network`, holding their test MSE in ppm². The mean predictor always uses the training target mean.

In [18]:

```
assert reg_frozen, "Record the regression choices before copper test evaluation."
reg_X_test, reg_y_test = reg_X[reg_test_idx], reg_y[reg_test_idx]
reg_mean_test_pred = np.full(len(reg_y_test), reg_y_mean)
```

```
# TODO 13
reg_kernel_test_pred = predict_kernel_raw(reg_X_test, reg_kernel_reloaded)
reg_nn_test_pred = predict_regression_nn_raw(reg_X_test, reg_nn_reloaded_model,
                                             reg_nn_reloaded_state)
reg_test_mse = {
    "Mean": regression_mse(reg_y_test, reg_mean_test_pred),
    "Kernel": regression_mse(reg_y_test, reg_kernel_test_pred),
    "Neural network": regression_mse(reg_y_test, reg_nn_test_pred)}

assert reg_kernel_test_pred.shape == reg_nn_test_pred.shape == reg_y_test.shape
assert set(reg_test_mse) == {"Mean", "Kernel", "Neural network"}
for name, value in reg_test_mse.items():
    assert np.isfinite(value) and value >= 0
    print(f"{name:15s} test MSE={value:14.1f} ppm²  RMSE={np.sqrt(value):8.1f} ppm")
candidate_summaries["regression_test"] = {
    name: {"mse_ppm2": float(value), "rmse_ppm": float(np.sqrt(value))}
    for name, value in reg_test_mse.items()}
```

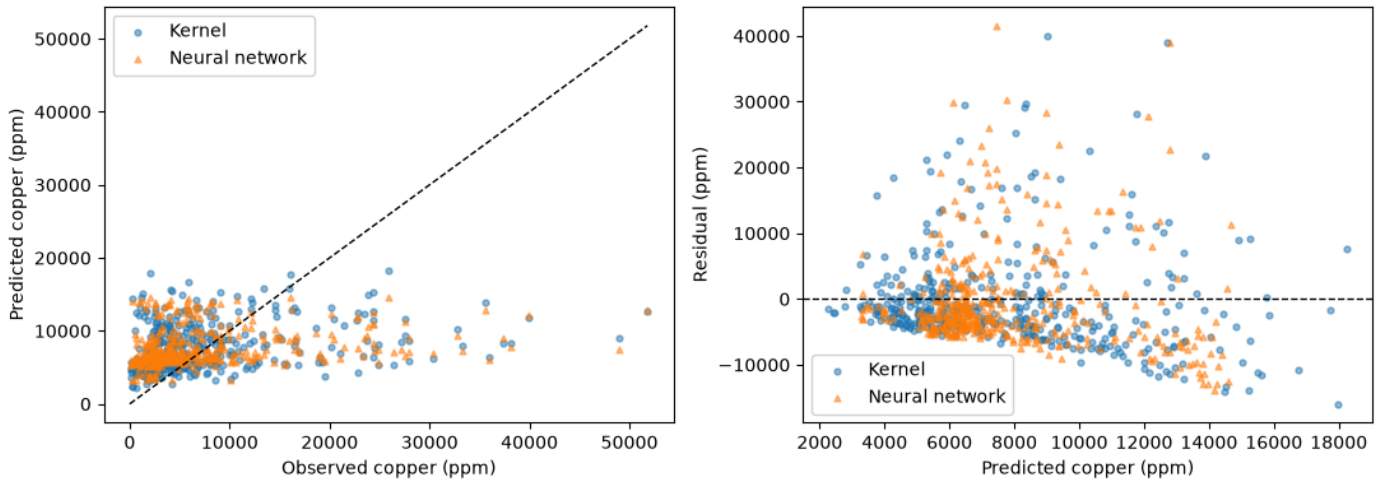
```
Mean            test MSE=    60807273.9 ppm²  RMSE=  7797.9 ppm
Kernel          test MSE=    56633521.8 ppm²  RMSE=  7525.5 ppm
Neural network  test MSE=    57977350.8 ppm²  RMSE=  7614.3 ppm
```

TODO 14: complete the two diagnostic plots for both selected models. In `axes[0]`, plot observed copper on the horizontal axis and predicted copper on the vertical axis. In `axes[1]`, plot prediction on the horizontal axis and residual (`observed - predicted`) on the vertical axis. Label each model.

In [19]:

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
# TODO 14
for label, prediction, marker in [("Kernel", reg_kernel_test_pred, "o"),
                                  ("Neural network", reg_nn_test_pred, "^")]:
    axes[0].scatter(reg_y_test, prediction, s=12, alpha=0.5, marker=marker,
label=label)
    axes[1].scatter(prediction, reg_y_test - prediction, s=12, alpha=0.5,
marker=marker,
                    label=label)

reg_plot_low = min(0., reg_y_test.min(), reg_kernel_test_pred.min(),
reg_nn_test_pred.min())
reg_plot_high = max(reg_y_test.max(), reg_kernel_test_pred.max(),
reg_nn_test_pred.max())
axes[0].plot([reg_plot_low, reg_plot_high], [reg_plot_low, reg_plot_high], "k--",
linewidth=1)
axes[0].set(xlabel="Observed copper (ppm)", ylabel="Predicted copper (ppm)")
axes[1].axhline(0., color="black", linestyle="--", linewidth=1)
axes[1].set(xlabel="Predicted copper (ppm)", ylabel="Residual (ppm)")
for ax in axes:
    ax.legend()
fig.tight_layout()
plt.show()
```



R2 — Compare validation and test errors. Use the TODO 13 results, your R1(a) answers, and the two diagnostic plots from TODO 14.

(a) Report test RMSE in ppm for the mean baseline, kernel, and neural network. List the three methods from lowest to highest validation RMSE, then from lowest to highest test RMSE. State whether the ordering changed; report any ties.

(b) Describe one error pattern visible for either model. Name the model and plot, and say where the pattern occurs, for example at high observed copper values or at large predicted values. Explain what the plotted points or residual signs show about its errors. If no systematic pattern is clear, describe the observed scatter. Do not retune the models.

Answer:

(a) Test RMSE: mean baseline 7797.9 ppm, kernel 7525.5 ppm, neural network 7614.3 ppm.

Validation RMSE from lowest to highest: kernel (8324.4), neural network (8422.7), mean baseline (8677.0). Test RMSE from lowest to highest: kernel (7525.5), neural network (7614.3), mean baseline (7797.9). The ordering did **not** change, and there are no ties. Both models beat the mean baseline, but only by a modest margin (the kernel's test RMSE is about 3.5% below the baseline). All three test RMSEs are lower than their validation RMSEs, which reflects the different holes in the two sets.

(b) Both models **under-predict the high-grade samples** (shown in the observed-versus-predicted plot, `axes[0]`, and the residual plot, `axes[1]`). In `axes[0]` the points with observed copper above about 20,000 ppm lie far below the dashed line: predictions stay roughly between 5,000 and 15,000 ppm while observed values reach about 52,000 ppm. In `axes[1]` (prediction on the horizontal axis) the largest residuals are large and positive, up to about +40,000 ppm, at predicted values of about 5,000 to 13,000 ppm, meaning observed copper was much higher than predicted. At the largest predicted values (above about 12,000 ppm) the residuals are mostly negative, i.e. over-prediction, and there is a dense group of small negative residuals at predicted values of 4,000 to 7,000 ppm. The predictions are compressed toward the mean, so neither model captures high-grade peaks at held-out holes.

R3 — Read the spatial prediction maps. Use the supplied two-panel prediction slice, where both models use the same x/y grid, depth, and color scale.

(a) Compare how gradually or abruptly predicted copper changes across the two maps. Identify one area where their predictions differ, using approximate x/y coordinates or a clear location such as the upper-left corner. State which model predicts higher copper there. If the maps look similar, report that and describe any visible difference. If no difference is clearly visible, state that.

(b) The white dots show x/y projections of samples that can have different z coordinates. Does a dense group of dots guarantee many samples near the displayed slice in all three coordinates? Explain. Why would few nearby measured samples make a local prediction less well supported?

(c) The prediction grid has no measured copper targets. Can you decide which model is more accurate from map smoothness alone? Explain which available result compares predictions with actual held-out measurements, and why it does not give an error for every point on this grid.

Answer:

(a) The **kernel** map changes more abruptly: it has a bright localized high-grade zone around x ≈ -800 to -600 m, y ≈ 0 to 150 m (about 14,000 ppm), and low patches in the upper right and lower right. The **neural network** map is smoother, with gradual broad gradients. One area where they differ is around x ≈ -800 m, y ≈ 80 m: the kernel predicts higher copper (about 14,000 ppm) than the network (about 9,000 ppm). The opposite occurs in the lower-left corner (around x ≈ -1000 m, y ≈ -400 m), where the network predicts higher copper than the kernel.

(b) No. The white dots are x/y projections of samples that can lie at any depth, so a dense group of dots may be many samples far above or below the displayed slice (z ≈ 96.6 m), and not close to it in all three coordinates. If few measured samples are near a location, the prediction there rests on distant samples (kernel) or on extrapolation (network), so it is less well supported and more uncertain.

(c) No. Smoothness is not accuracy: a smooth map can be wrong and a more variable one can be right. The result that compares predictions with actual held-out measurements is the test evaluation (TODO 13–14), which uses the measured copper at the test sample locations. The grid has no measured copper values, so there is no truth to compute an error for every grid point.

D. MNIST: pixels, scores, and probabilities

Use the supplied **5,000 training**, **1,000 validation**, and **2,000 test** images. Each digit contributes 500, 100, and 200 images respectively. The training and validation images come from the official MNIST training split; the test images come from its separate official test split. The fixed subset indices are included in `mnist_small.npz`.

First detect **digit 5 versus every other digit**. Then recognize all ten digits. Both tasks use the same image indices. Keep the MNIST test arrays closed until **E. MNIST: final evaluation**, after the MNIST choices and R4 are complete.

Divide the supplied uint8 pixel values by 255. This fixed transformation uses no fitted statistics. Do not standardize each image separately.

TODO 15: load only the training and validation arrays from `mnist_small.npz`. Define float32 input tensors `mnist_X_train` and `mnist_X_val`, each with 784 columns. Define integer digit-label

tensors `mnist_y_train` and `mnist_y_val`. Define floating binary targets `mnist_b_train` and `mnist_b_val`, each with one column, where 1 means digit 5.

In [20]:

```
with np.load(DATA_DIR / "mnist_small.npz", allow_pickle=False) as data:
    # TODO 15: training and validation arrays only.
    raw_train, raw_val = data["x_train"], data["x_val"]
    mnist_X_train = torch.tensor(raw_train.reshape(len(raw_train),
784).astype(np.float32) / 255.0)
    mnist_X_val = torch.tensor(raw_val.reshape(len(raw_val), 784).astype(np.float32) /
255.0)
    mnist_y_train = torch.tensor(data["y_train"].astype(np.int64))
    mnist_y_val = torch.tensor(data["y_val"].astype(np.int64))
    mnist_b_train = (mnist_y_train == 5).to(torch.float32).unsqueeze(1)
    mnist_b_val = (mnist_y_val == 5).to(torch.float32).unsqueeze(1)
```

In [21]:

```
assert mnist_X_train.shape == (5000, 784)
assert mnist_X_val.shape == (1000, 784)
assert mnist_X_train.dtype == mnist_X_val.dtype == torch.float32
assert 0 <= mnist_X_train.min() and mnist_X_train.max() <= 1
assert 0 <= mnist_X_val.min() and mnist_X_val.max() <= 1
assert mnist_y_train.dtype == mnist_y_val.dtype == torch.int64
assert mnist_b_train.shape == (5000, 1)
assert mnist_b_val.shape == (1000, 1)
assert mnist_b_train.dtype == mnist_b_val.dtype == torch.float32
assert torch.equal(torch.bincount(mnist_y_train), torch.full((10,), 500))
assert torch.equal(torch.bincount(mnist_y_val), torch.full((10,), 100))
assert int(mnist_b_train.sum()) == 500 and int(mnist_b_val.sum()) == 100
print("MNIST train / validation:", len(mnist_X_train), len(mnist_X_val))
print("Binary positive counts:", int(mnist_b_train.sum()), int(mnist_b_val.sum()))
```

```
MNIST train / validation: 5000 1000
Binary positive counts: 500 100
```

S5 — Inputs and binary labels. Refer to the transformations in TODO 15.

(a) What do one row and the 784 columns of `mnist_X_train` represent? Explain how 784 follows from the original image shape.

(b) What does the single value in each row of `mnist_b_train` represent? State which original digit maps to 1 and which digits map to 0.

(c) Does changing an image's label to 5-versus-rest change which image it is or which split it belongs to? Explain why the existing split can be reused.

Answer:

(a) One row is one image. The 784 columns are its pixel intensities, scaled to [0, 1] by dividing by 255. The original image is 28 × 28 pixels, and 28 × 28 = 784, so flattening the image row by row gives 784 values.

(b) The single value in each row of `mnist_b_train` is the binary target for "is this image digit 5?". The original digit 5 maps to 1; digits 0, 1, 2, 3, 4, 6, 7, 8 and 9 map to 0.

(c) No. Relabelling changes only the target, not which image it is and not which split it belongs to. The same 5000 training and 1000 validation images stay in the same sets, so the existing split can be reused: it still keeps training, validation and test images separate.

TODO 16: implement `binary_metrics` using NumPy. Return one Python `dict` **with exactly these seven keys**, spelled as shown:

- `tn` : number of true negatives.
- `fp` : number of false positives.
- `fn` : number of false negatives.
- `tp` : number of true positives.
- `accuracy` : fraction of all predictions that are correct.
- `precision` : precision for the positive class (digit 5).
- `recall` : recall for the positive class (digit 5).

The four counts must be integers. Return accuracy, precision, and recall as numbers between 0 and 1, rather than percentages. If the denominator of precision or recall is zero, return 0 for that metric.

In [22]:

```
def binary_metrics(y_true, y_pred):
    y_true = np.asarray(y_true).reshape(-1).astype(int)
    y_pred = np.asarray(y_pred).reshape(-1).astype(int)
    # TODO 16: return a dict with tn, fp, fn, tp, accuracy, precision, recall.
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    accuracy = (tn + tp) / (tn + fp + fn + tp)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    return {"tn": tn, "fp": fp, "fn": fn, "tp": tp,
            "accuracy": float(accuracy), "precision": float(precision), "recall":
float(recall)}
```

In [23]:

```
check = binary_metrics([1, 1, 0, 0, 0], [1, 0, 1, 0, 0])
assert [check[k] for k in ("tn", "fp", "fn", "tp")] == [2, 1, 1, 1]
assert np.isclose(check["accuracy"], 0.6)
assert all(np.isclose(check[k], 0.5) for k in ("precision", "recall"))
asymmetric = binary_metrics([1, 1, 1, 0, 0, 0], [1, 0, 0, 1, 0, 0])
assert [asymmetric[k] for k in ("tn", "fp", "fn", "tp")] == [2, 1, 2, 1]
assert np.isclose(asymmetric["precision"], 1 / 2)
assert np.isclose(asymmetric["recall"], 1 / 3)
assert np.isclose(asymmetric["accuracy"], 1 / 2)
no_alerts = binary_metrics([0, 0, 1], [0, 0, 0])
assert no_alerts["precision"] == no_alerts["recall"] == 0
no_positives = binary_metrics([0, 0], [1, 0])
assert no_positives["recall"] == 0
mnist_binary_baseline = binary_metrics(
    mnist_b_val.numpy(), np.zeros(len(mnist_b_val), dtype=int))
print("Validation baseline: always predict not-5", mnist_binary_baseline)
```

```
Validation baseline: always predict not-5 {'tn': 900, 'fp': 0, 'fn': 100, 'tp': 0,
'accuracy': 0.9, 'precision': 0.0, 'recall': 0.0}
```

Construct the three classifier architectures

TODO 17: implement `make_mnist_classifier(name)` to construct the requested network and its matching loss function. The input `name` is one of the three strings in the table below.

Each call must create and return a **two-item tuple `(model, loss)`**:

- `model`: a new, **untrained `nn.Sequential` object** with the specified layers.
- `loss`: the corresponding **loss-function object**, created by calling `nn.BCEWithLogitsLoss()` or `nn.CrossEntropyLoss()` as specified below. The numerical loss will be calculated later from model outputs and targets.

Create both objects inside the function and return them. **Training these networks is the task in TODO 19**, where this function will be called.

`name`	Architecture to construct	Loss-function object
`binary`	784 → 1, linear	`nn.BCEWithLogitsLoss()`
`linear10`	784 → 10, linear	`nn.CrossEntropyLoss()`
`mlp10`	784 → 64 → 10, ReLU after the hidden layer	`nn.CrossEntropyLoss()`

Configure each network so that calling it on a batch of images later produces raw scores (logits). Its final layer is linear, with no sigmoid or softmax layer after it.

In [24]:

```
def make_mnist_classifier(name):
    # TODO 17: return (new untrained network, matching loss-function object).
    if name == "binary":
        return nn.Sequential(nn.Linear(784, 1)), nn.BCEWithLogitsLoss()
    elif name == "linear10":
        return nn.Sequential(nn.Linear(784, 10)), nn.CrossEntropyLoss()
    elif name == "mlp10":
        return (nn.Sequential(nn.Linear(784, 64), nn.ReLU(), nn.Linear(64, 10)),
                nn.CrossEntropyLoss())
    raise ValueError(f"Unknown model name: {name}")
```

In [25]:

```
for name, width, loss_type in [("binary", 1, nn.BCEWithLogitsLoss),
                               ("linear10", 10, nn.CrossEntropyLoss),
                               ("mlp10", 10, nn.CrossEntropyLoss)]:
    model_check, loss_check = make_mnist_classifier(name)
    assert model_check(torch.zeros(3, 784)).shape == (3, width)
    assert isinstance(loss_check, loss_type)
    expected_sizes = {"binary": [(784, 1)], "linear10": [(784, 10)],
                      "mlp10": [(784, 64), (64, 10)]}[name]
    assert [(m.in_features, m.out_features) for m in model_check.modules()
            if isinstance(m, nn.Linear)] == expected_sizes
    assert sum(isinstance(m, nn.ReLU) for m in model_check.modules()) == (
        1 if name == "mlp10" else 0)
```

```
    assert not any(isinstance(m, (nn.Sigmoid, nn.Softmax))
                   for m in model_check.modules())
del model_check, loss_check
```

S6 — Match outputs, targets, and losses. Refer to TODO 15 and TODO 17.

(a) For a full batch of **128 images**, complete the fields under **Answer (a)** below. Replace each `[fill in]` with the raw model output shape, target shape, target dtype, or loss-function name requested. The ten-class entry applies to both `linear10` and `mlp10`.

(b) Should you apply sigmoid or softmax to the model outputs before passing them to these loss functions? Explain what each loss already computes.

(c) When you want predicted probabilities instead of a training loss, which transformation should you apply to the binary outputs and to the ten-class outputs? State which axis the ten-class transformation uses.

Answer:

(a)

Binary model

- Raw model output shape: (128, 1)
- Target shape: (128, 1)
- Target dtype: torch.float32
- Loss-function name: nn.BCEWithLogitsLoss

Ten-class models (`linear10` and `mlp10`)

- Raw model output shape: (128, 10)
- Target shape: (128,)
- Target dtype: torch.int64
- Loss-function name: nn.CrossEntropyLoss

(b) No. Both losses expect raw scores (logits). `BCEWithLogitsLoss` applies the sigmoid internally and combines it with binary cross-entropy in a numerically stable way. `CrossEntropyLoss` applies log-softmax internally and then the negative log-likelihood of the true class. Applying sigmoid or softmax first would apply it twice, which distorts the loss and gradients.

(c) For the binary outputs apply `torch.sigmoid(logits)`. For the ten-class outputs apply `torch.softmax(logits, dim=1)`, i.e. along axis 1, across the 10 classes, so each image's ten probabilities sum to 1.

TODO 18: implement `make_mnist_loaders`. Return training and validation loaders with batch size 128. Select binary or digit targets as appropriate. Shuffle the training data and do not shuffle the validation data. Use the supplied `generator` when creating the training loader by passing `generator=generator` to `DataLoader`. Keep the final short batch.

In [26]:

```
def make_mnist_loaders(name, seed=3230):  # Keep the default seed at 3230.
    # Supplied: generator for reproducible training-data shuffling.
    generator = torch.Generator().manual_seed(seed)  # Do not change the supplied seed.
    # TODO 18: pair the correct targets with inputs, then create two loaders.
    if name == "binary":
        train_targets, val_targets = mnist_b_train, mnist_b_val
    else:
        train_targets, val_targets = mnist_y_train, mnist_y_val
    train_loader = DataLoader(TensorDataset(mnist_X_train, train_targets),
batch_size=128,
                              shuffle=True, num_workers=0, generator=generator)
    val_loader = DataLoader(TensorDataset(mnist_X_val, val_targets), batch_size=128,
                            shuffle=False, num_workers=0)
    return train_loader, val_loader
```

In [27]:

```
check_train_a, check_val = make_mnist_loaders("binary")
check_train_b, _ = make_mnist_loaders("binary")
batch_a = next(iter(check_train_a))
batch_b = next(iter(check_train_b))
assert torch.equal(batch_a[0], batch_b[0]) and torch.equal(batch_a[1], batch_b[1])
assert batch_a[0].shape == (128, 784) and batch_a[1].shape == (128, 1)
assert sum(len(xb) for xb, _ in check_train_a) == 5000
assert torch.equal(torch.cat([yb for _, yb in check_val]), mnist_b_val)
_, check_val10 = make_mnist_loaders("mlp10")
assert torch.equal(torch.cat([yb for _, yb in check_val10]), mnist_y_val)
del check_train_a, check_train_b, check_val, check_val10, batch_a, batch_b
```

Fixed training budget

TODO 19: train the three models once each. For every model, reset `torch.manual_seed(3230)`, build a fresh model and independent loaders, and reuse `fit_network` from the regression section. Use **20 epochs**, SGD with **learning rate 0.1**, and **no scheduler**.

Fill the three dictionaries below. **Each dictionary must contain exactly these three string keys:** `"binary"`, `"linear10"`, and `"mlp10"`. For each model name `name`, store the following value:

- **`mnist_models[name]`:** the trained model object after calling `fit_network`. Its weights must be restored to the epoch with the lowest validation loss. Store the model object itself, not its `state_dict` or its predictions.
- **`mnist_criteria[name]`:** the loss-function object returned by `make_mnist_classifier(name)`: an `nn.BCEWithLogitsLoss()` object for `"binary"`, and an `nn.CrossEntropyLoss()` object for both `"linear10"` and `"mlp10"`. Store the object, not a computed loss value.
- **`mnist_histories[name]`:** the complete history dictionary returned by `fit_network` for that model. It contains `"train_loss"`, `"val_loss"`, and `"lr"` (each a list of 20 values, one per epoch), and `"best_epoch"` (the selected epoch number from 1 to 20).

`fit_network` updates the supplied model in place, restores its best validation weights, and returns the history dictionary. Its recorded training and validation losses evaluate fixed weights at the end of each epoch. Do not add architecture, learning-rate, or seed searches.

In [28]:

```
mnist_names = ("binary", "linear10", "mlp10")
mnist_models, mnist_criteria, mnist_histories = {}, {}, {}
for name in mnist_names:
    # TODO 19: fit one fresh model with the prescribed budget and store results.
    torch.manual_seed(3230)
    model, criterion = make_mnist_classifier(name)
    train_loader, val_loader = make_mnist_loaders(name)
    history = fit_network(model, train_loader, val_loader, criterion,
                          epochs=20, lr=0.1, step_size=None)
    mnist_models[name] = model
    mnist_criteria[name] = criterion
    mnist_histories[name] = history
```

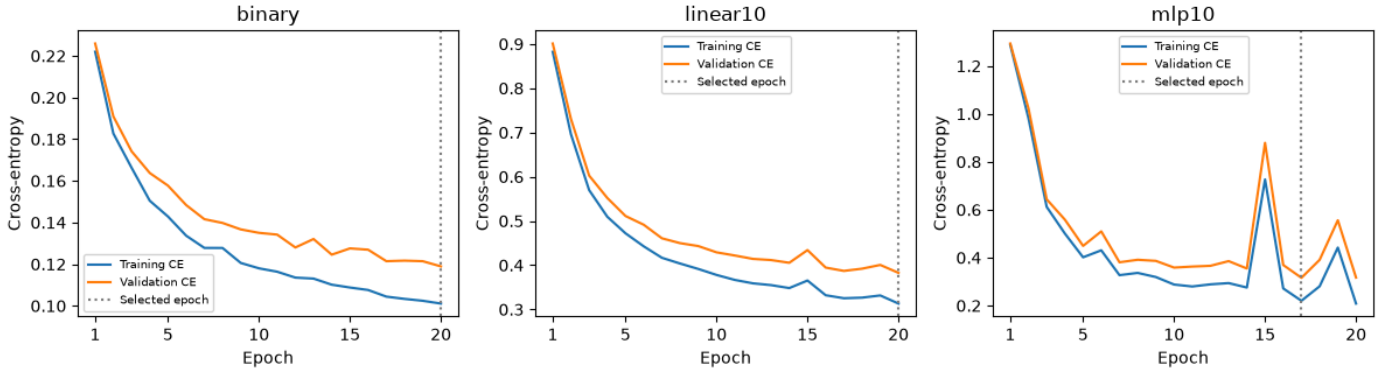
In [29]:

```
for name in mnist_names:
    history = mnist_histories[name]
    assert len(history["train_loss"]) == len(history["val_loss"]) == 20
    assert np.isfinite(history["train_loss"]).all()
    assert np.isfinite(history["val_loss"]).all()
    _, val_loader = make_mnist_loaders(name)
    restored_ce = evaluate_loss(mnist_models[name], val_loader, mnist_criteria[name])
    assert np.isclose(restored_ce, min(history["val_loss"]), atol=1e-6)
```

TODO 20: add training and validation CE curves to each supplied axis. Mark the selected epoch with a vertical line. Label epochs from 1 to 20, and include a legend. Compare the two ten-class models using their own curves; binary CE and ten-class CE concern different response tasks.

In [30]:

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.4))
for ax, name in zip(axes, mnist_names):
    history = mnist_histories[name]
    # TODO 20: draw and label the two curves and selected epoch.
    epochs_axis = np.arange(1, 21)
    ax.plot(epochs_axis, history["train_loss"], label="Training CE")
    ax.plot(epochs_axis, history["val_loss"], label="Validation CE")
    ax.axvline(history["best_epoch"], color="gray", linestyle=":", label="Selected
epoch")
    ax.set_xticks([1, 5, 10, 15, 20])
    ax.set(xlabel="Epoch", ylabel="Cross-entropy", title=name)
    ax.legend(fontsize=7)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "mnist_learning_curves.png", dpi=150)
plt.show()
```



Validation fixes the decisions

TODO 21: use the restored binary model to calculate validation probabilities `mnist_p_val`. For thresholds `0.00, 0.05, ..., 0.95`, compute label-based metrics with `binary_metrics`. Store one dictionary per threshold in `mnist_threshold_rows`, adding its value under `threshold`.

Select `mnist_final_t` in two steps:

1. Keep only thresholds with **validation recall >= 0.80**.
2. Among these thresholds, choose the one with the **highest validation precision**. If several have the same highest precision, choose the first one in the supplied ascending order (the smallest threshold).

Use the unrounded recall and precision values for selection. The rule predicts 5 when `probability >= threshold`, so threshold `0.00` predicts every image as 5 and has recall 1.00. It ensures that at least one candidate meets the recall requirement. Do not retrain the network.

In [31]:

```
mnist_thresholds = np.arange(20, dtype=float) / 20
# TODO 21: probabilities, all 20 metric rows, then highest precision among recall >=
0.80.
binary_model = mnist_models["binary"]
binary_model.eval()
with torch.no_grad():
    mnist_p_val = torch.sigmoid(binary_model(mnist_X_val)).numpy().reshape(-1)
mnist_val_labels = mnist_b_val.numpy().reshape(-1).astype(int)
mnist_threshold_rows = []
for threshold in mnist_thresholds:
    row = binary_metrics(mnist_val_labels, (mnist_p_val >= threshold).astype(int))
    row["threshold"] = float(threshold)
    mnist_threshold_rows.append(row)
feasible_rows = [row for row in mnist_threshold_rows if row["recall"] >= 0.80]
# max() keeps the first (smallest) threshold among exactly tied precisions.
mnist_final_t = max(feasible_rows, key=lambda row: row["precision"])["threshold"]
```

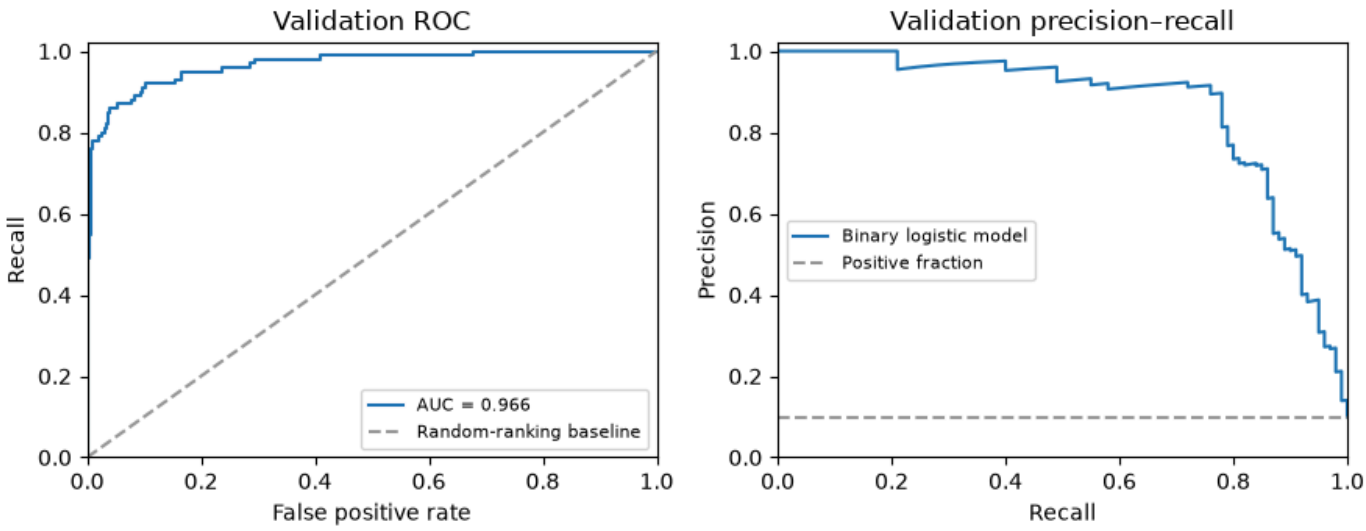
In [32]:

```
assert mnist_p_val.shape == (1000,) and np.isfinite(mnist_p_val).all()
assert np.all((0 <= mnist_p_val) & (mnist_p_val <= 1))
assert len(mnist_threshold_rows) == 20
assert np.array_equal([row["threshold"] for row in mnist_threshold_rows],
                      np.arange(20, dtype=float) / 20)
assert mnist_threshold_rows[0]["recall"] == 1.0
```

```
check_feasible_rows = [row for row in mnist_threshold_rows if row["recall"] >= 0.80]
assert mnist_final_t == check_feasible_rows[
    int(np.argmax([row["precision"] for row in check_feasible_rows]))]["threshold"]
print("Threshold | accuracy | precision | recall | alerts")
for row in mnist_threshold_rows:
    print(f'{row["threshold"]:9.2f} | {row["accuracy"]:8.3f} | '
          f'{row["precision"]:9.3f} | {row["recall"]:6.3f} | '
          f'{row["tp"] + row["fp"]:4d}')
print("Selected threshold:", mnist_final_t)

from sklearn.metrics import roc_curve, precision_recall_curve, roc_auc_score
binary_val_y = mnist_b_val.numpy().ravel().astype(int)
fpr, tpr, _ = roc_curve(binary_val_y, mnist_p_val)
precision, recall, _ = precision_recall_curve(binary_val_y, mnist_p_val)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
axes[0].plot(fpr, tpr, label=f'AUC = {roc_auc_score(binary_val_y, mnist_p_val):.3f}')
axes[0].plot([0, 1], [0, 1], "--", color="0.6", label="Random-ranking baseline")
axes[0].set(xlabel="False positive rate", ylabel="Recall", title="Validation ROC")
axes[1].plot(recall, precision, label="Binary logistic model")
axes[1].axhline(binary_val_y.mean(), linestyle="--", color="0.6",
                label="Positive fraction")
axes[1].set(xlabel="Recall", ylabel="Precision", title="Validation precision-recall")
for ax in axes:
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1.02)
    ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "mnist_validation_roc_pr.png", dpi=150)
plt.show()
```

```
Threshold | accuracy | precision | recall | alerts
     0.00 |    0.100 |     0.100 |  1.000 | 1000
     0.05 |    0.839 |     0.378 |  0.950 |  251
     0.10 |    0.905 |     0.514 |  0.900 |  175
     0.15 |    0.932 |     0.613 |  0.870 |  142
     0.20 |    0.950 |     0.708 |  0.850 |  120
     0.25 |    0.955 |     0.767 |  0.790 |  103
     0.30 |    0.960 |     0.806 |  0.790 |   98
     0.35 |    0.965 |     0.857 |  0.780 |   91
     0.40 |    0.968 |     0.905 |  0.760 |   84
     0.45 |    0.965 |     0.922 |  0.710 |   77
     0.50 |    0.963 |     0.920 |  0.690 |   75
     0.55 |    0.953 |     0.908 |  0.590 |   65
     0.60 |    0.953 |     0.921 |  0.580 |   63
     0.65 |    0.949 |     0.930 |  0.530 |   57
     0.70 |    0.947 |     0.961 |  0.490 |   51
     0.75 |    0.943 |     0.957 |  0.450 |   47
     0.80 |    0.937 |     0.974 |  0.380 |   39
     0.85 |    0.930 |     0.969 |  0.310 |   32
     0.90 |    0.921 |     0.957 |  0.220 |   23
     0.95 |    0.912 |     1.000 |  0.120 |   12
Selected threshold: 0.2
```



S7 — Thresholds and binary evaluation. Use the baseline output after TODO 16 and the threshold/ROC code after TODO 21.

(a) The all-not-5 rule has 90% validation accuracy. What is its recall for digit 5? Use this to explain why accuracy alone does not show useful detection of digit 5.

(b) Keep the model and validation images fixed, and lower the decision threshold. State whether the model parameters, predicted probabilities, and validation CE change. Can an image already predicted as 5 become predicted as not-5? Explain what this implies for recall, and whether precision is guaranteed to increase.

(c) Why does the supplied ROC AUC calculation use `mnist_p_val` rather than the 0/1 predictions at one threshold? Explain what information about the ordering of images by their scores would be lost by using those labels.

Answer:

(a) The all-not-5 rule has recall 0, because it finds none of the 100 validation 5s (tp = 0, fn = 100). Its 90% accuracy comes entirely from the 900 other digits, since only 10% of images are 5s. Accuracy therefore does not show useful detection of digit 5: a rule that never detects a 5 still scores high.

(b) Lowering the threshold changes **none** of the model parameters, the predicted probabilities, or the validation CE; the threshold is only applied afterwards to turn probabilities into labels. An image already predicted as 5 **cannot** become not-5, because if probability ≥ the old threshold it is also ≥ the lower one. So true positives never decrease and recall cannot decrease. Precision is **not** guaranteed to increase: more images are flagged and false positives usually rise (in the table, precision falls from 0.920 at 0.50 to 0.378 at 0.05).

(c) ROC AUC measures how well the scores **rank** true 5s above non-5s across all possible thresholds, so it needs the probabilities `mnist_p_val`. The 0/1 predictions at one threshold collapse every score to one of two values, losing the ordering of images within each group and giving only a single point of the ROC curve.

TODO 22: evaluate both restored ten-class models on validation images. Store each model's CE and accuracy in `mnist_multiclass_val_results`. Use `{model_name: {"ce": float,`

`"accuracy": float}}` as the dictionary schema. Choose `mnist_selected_multiclass` by the lower validation CE, taking `linear10` in an exact tie. Use argmax for class decisions.

In [33]:

```
mnist_multiclass_val_results = {}
# TODO 22: validation results and the chosen ten-class model name.
for name in ("linear10", "mlp10"):
    model, criterion = mnist_models[name], mnist_criteria[name]
    model.eval()
    with torch.no_grad():
        logits = model(mnist_X_val)
        val_ce = float(criterion(logits, mnist_y_val))
        val_accuracy = float((logits.argmax(dim=1) ==
mnist_y_val).to(torch.float32).mean())
    mnist_multiclass_val_results[name] = {"ce": val_ce, "accuracy": val_accuracy}
mnist_selected_multiclass = ("linear10"
    if mnist_multiclass_val_results["linear10"]["ce"] <=
mnist_multiclass_val_results["mlp10"]["ce"]
    else "mlp10")
```

In [34]:

```
assert set(mnist_multiclass_val_results) == {"linear10", "mlp10"}
assert mnist_selected_multiclass in ("linear10", "mlp10")
mnist_selection = {
    "seed": 3230, "epochs": 20, "batch_size": 128, "lr": 0.1,
    "positive_digit": 5, "binary_threshold": float(mnist_final_t),
    "selected_multiclass": mnist_selected_multiclass,
    "best_epochs": {name: int(mnist_histories[name]["best_epoch"]) for name in
mnist_names},
    "multiclass_validation": mnist_multiclass_val_results,
    "pixel_divisor": 255.0, "input_shape": [28, 28],
    "architectures": {"binary": [784, 1], "linear10": [784, 10],
                      "mlp10": [784, 64, 10]},
}
with open(OUTPUT_DIR / "mnist_selection.json", "w", encoding="utf-8") as handle:
    json.dump(mnist_selection, handle, indent=2)
print(json.dumps(mnist_selection, indent=2))
```


```
{
  "seed": 3230,
  "epochs": 20,
  "batch_size": 128,
  "lr": 0.1,
  "positive_digit": 5,
  "binary_threshold": 0.2,
  "selected_multiclass": "mlp10",
  "best_epochs": {
    "binary": 20,
    "linear10": 20,
    "mlp10": 17
  },
  "multiclass_validation": {
    "linear10": {
      "ce": 0.38267555832862854,
      "accuracy": 0.890999972820282
    },
    "mlp10": {
      "ce": 0.31597548723220825,
```

```
      "accuracy": 0.9079999923706055
    }
  },
  "pixel_divisor": 255.0,
  "input_shape": [
    28,
    28
  ],
  "architectures": {
    "binary": [
      784,
      1
    ],
    "linear10": [
      784,
      10
    ],
    "mlp10": [
      784,
      64,
      10
    ]
  }
}
```


S8 — Convert probabilities to labels. Refer to TODO 21–22.

(a) Using the selected threshold `mnist_final_t`, state the condition for predicting an image as digit 5. What prediction is made when this condition is not satisfied?

(b) Suppose all ten class probabilities for an image are below 0.5. Would selecting every class whose probability is at least 0.5 return a digit? Would argmax still return one? Explain why argmax fits a task that requires exactly one digit label per image.

(c) Suppose two models predict exactly the same digit label for every validation image, but assign different probabilities to the true labels. Must their validation accuracy be the same? Must their validation CE be the same? Explain what information each metric uses.

Answer:

(a) Predict digit 5 when `mnist_p_val` (the predicted probability) ≥ `mnist_final_t` (0.2). When this condition is not satisfied, predict not-5 (label 0).

(b) No. If all ten probabilities are below 0.5, selecting every class with probability at least 0.5 returns **no** digit (an empty selection). Argmax still returns one, the class with the largest probability. Argmax fits a task that needs exactly one label per image, because it always gives exactly one class (apart from exact ties), consistent with the ten classes being mutually exclusive.

(c) Accuracy: yes, it must be the same, because it uses only whether each predicted label equals the true label. CE: no, it need not be the same, because CE uses the probability assigned to the true class (-log of that probability), so models with different probabilities can have different CE even with identical labels.

R4 — Record the MNIST choices before testing. Use the threshold table from TODO 21, `mnist_selection` after TODO 22, and the validation plots.

(a) Complete the three supplied blocks under **Answer (a)**: **Binary model**, **Ten-class models**, and **Selection rules**. Replace each `[fill in]` with the requested number, model name, or metric name. Use your run's validation results, and write accuracy, precision, and recall as proportions between 0 and 1.

(b) In the selected ten-class model's learning-curve panel from TODO 20, does validation CE continue to decrease as training CE decreases, or do the curves diverge during any interval? Describe one observed interval using approximate epoch numbers. Report the actual pattern; a divergence is not required.

(c) Choose **one** validation plot: ROC or precision–recall. State which plot you chose and whether the observed curve is mostly above, near, or below its plotted reference line (the ROC diagonal or the PR line at 0.10). Explain what that comparison indicates about this model's validation performance. One or two sentences are sufficient.

Complete this response before the MNIST test evaluation. Keep the recorded choices fixed after viewing test results.

Answer:

(a)

Binary model

- Selected epoch: 20
- Selected threshold: 0.20
- Validation accuracy at this threshold: 0.950
- Validation precision at this threshold: 0.708
- Validation recall at this threshold: 0.850

Ten-class models

- `linear10` validation CE: 0.3827
- `mlp10` validation CE: 0.3160
- Selected model name (`linear10` or `mlp10`): mlp10
- Selected epoch of this model: 17

Selection rules

- Checkpoint: minimize validation cross-entropy (CE loss).
- Binary threshold: validation recall must be at least 0.80; among those thresholds, maximize validation precision.
- Ten-class model: minimize validation cross-entropy (CE).

(b) For `mlp10`, training and validation CE fall together from epoch 1 to about epoch 10 (validation CE from about 1.29 to about 0.36). After that validation CE does not keep decreasing: it stays around 0.36 to 0.39, while training CE is lower and keeps drifting down (about 0.29 near epoch 10, about 0.21 at

epoch 20). In particular, from epoch 17 to epoch 20 training CE decreases (about 0.220 to 0.208) while validation CE does not improve (about 0.316 to 0.317), which is why epoch 17 is selected. Both curves also spike at epochs 15 and 19 (validation CE about 0.88 at epoch 15).

(c) I chose the **ROC** plot. The observed curve is far **above** the diagonal reference line (AUC = 0.966), which means that on validation data the model ranks true 5s above other digits much better than random ranking would, so it has useful discriminating power.

E. MNIST: final evaluation

Run this section only after R4 records the MNIST model, checkpoint, and threshold choices. There is no further fitting or model selection below.

TODO 23: load and transform the test images with the same fixed divide-by-255 rule. Define `mnist_X_test`, `mnist_y_test`, and binary targets `mnist_b_test`; retain `mnist_raw_test` for displaying errors. Calculate `mnist_p_test`, binary test CE `mnist_binary_test_ce`, and ROC AUC `mnist_binary_test_auc` using the restored binary model. Store label-based metrics in `mnist_binary_test_comparison` under `selected_threshold` and `fixed_0.5`.

In [35]:

```
assert mnist_selection, "Record the MNIST choices before MNIST test evaluation."
# TODO 23: fixed test evaluation of the binary model and both pre-specified rules.
with np.load(DATA_DIR / "mnist_small.npz", allow_pickle=False) as data:
    mnist_raw_test = data["x_test"].copy()
    test_digits = data["y_test"].astype(np.int64)
mnist_X_test = torch.tensor(mnist_raw_test.reshape(len(mnist_raw_test), 784)
                            .astype(np.float32) / 255.0)
mnist_y_test = torch.tensor(test_digits)
mnist_b_test = (mnist_y_test == 5).to(torch.float32).unsqueeze(1)

binary_model = mnist_models["binary"]
binary_model.eval()
with torch.no_grad():
    test_logits = binary_model(mnist_X_test)
    mnist_p_test = torch.sigmoid(test_logits).numpy().reshape(-1)
    mnist_binary_test_ce = float(mnist_criteria["binary"](test_logits, mnist_b_test))
mnist_test_labels = mnist_b_test.numpy().reshape(-1).astype(int)
mnist_binary_test_auc = float(roc_auc_score(mnist_test_labels, mnist_p_test))
mnist_binary_test_comparison = {}
for rule, threshold in [("selected_threshold", mnist_final_t), ("fixed_0.5", 0.5)]:
    row = binary_metrics(mnist_test_labels, (mnist_p_test >= threshold).astype(int))
    row["threshold"] = float(threshold)
    mnist_binary_test_comparison[rule] = row
```

In [36]:

```
assert mnist_X_test.shape == (2000, 784) and mnist_p_test.shape == (2000,)
assert torch.equal(torch.bincount(mnist_y_test), torch.full((10,), 200))
assert int(mnist_b_test.sum()) == 200
assert np.isfinite(mnist_binary_test_ce) and 0 <= mnist_binary_test_auc <= 1
for rule, row in mnist_binary_test_comparison.items():
    assert sum(row[k] for k in ("tn", "fp", "fn", "tp")) == 2000
    print(rule, row)
```

```
print("Shared binary test CE / AUC:", mnist_binary_test_ce, mnist_binary_test_auc)
selected = mnist_binary_test_comparison["selected_threshold"]
print("Selected-rule confusion matrix (true rows; predicted columns):")
print(np.array([[selected["tn"], selected["fp"]],
                [selected["fn"], selected["tp"]]]))
```

```
selected_threshold {'tn': 1724, 'fp': 76, 'fn': 26, 'tp': 174, 'accuracy': 0.949,
'precision': 0.696, 'recall': 0.87, 'threshold': 0.2}
fixed_0.5 {'tn': 1781, 'fp': 19, 'fn': 57, 'tp': 143, 'accuracy': 0.962, 'precision':
0.8827160493827161, 'recall': 0.715, 'threshold': 0.5}
Shared binary test CE / AUC: 0.11605827510356903 0.9630805555555556
Selected-rule confusion matrix (true rows; predicted columns):
[[1724   76]
 [  26  174]]
```

R5 — Compare the two fixed binary rules. Use the results printed after TODO 23. Both rules use the same trained binary model and test images.

(a) Complete the two supplied blocks under **Answer (a)**. Use `mnist_final_t` for the validation-selected threshold, and copy the test FP, FN, accuracy, precision, and recall from the TODO 23 output for each rule. Write accuracy, precision, and recall as proportions between 0 and 1. Then complete each comparison line by choosing **more**, **fewer**, or **the same number of**. Compare FN for missed digit-5 images, and FP for other digits incorrectly classified as 5.

(b) Explain a false positive and a false negative by stating the image's true class and the model's predicted class. Which threshold remains the final rule if 0.5 performs better on test, or if the selected rule's test recall is below 0.80? Explain why meeting the recall requirement on validation does not guarantee meeting it on test, and why test results cannot change the threshold choice.

(c) Of the reported ROC AUC and CE, which evaluates how images are ranked by their scores, and which evaluates probabilities assigned to observed labels? Explain the distinction briefly; no additional calculation is needed.

Answer:

(a)

Validation-selected rule

- Threshold: 0.20
- Test FP: 76
- Test FN: 26
- Test accuracy: 0.949
- Test precision: 0.696
- Test recall: 0.870

Fixed 0.5 rule

- Threshold: 0.50
- Test FP: 19
- Test FN: 57
- Test accuracy: 0.962
- Test precision: 0.883

- Test recall: 0.715

Comparison with the fixed 0.5 rule

- The validation-selected rule misses **fewer** digit-5 images (26 versus 57).
- The validation-selected rule incorrectly classifies **more** other-digit images as 5 (76 versus 19).

(b) A **false positive** is an image whose true class is another digit (for example a 3 or a 0) that the model predicts as 5. A **false negative** is an image whose true class is 5 that the model predicts as not-5. The final rule remains the **validation-selected threshold 0.20** in both situations: even if 0.5 performs better on test (here 0.5 has higher accuracy and precision but lower recall) and even if the selected rule's test recall were below 0.80 (here it is 0.870, so the requirement is met). Meeting recall ≥ 0.80 on validation does not guarantee it on test, because the validation set has only 100 positive images and the test set contains different images, so recall varies from sample to sample. Test results cannot change the threshold: choosing it with test data would use the test set for selection, so it would no longer be an unbiased, unseen check of the final rule.

(c) **ROC AUC** (0.963) evaluates how the images are **ranked** by their scores, independent of any single threshold. **CE** (0.116) evaluates the **probabilities assigned to the observed labels**: it penalizes assigning low probability to the true label, especially with confidence, so it depends on the probability values and not just their order.

TODO 24: evaluate both restored ten-class models on the test images. Use evaluation mode and disable gradient recording. Calculate CE from the raw logits and integer targets using each model's loss-function object. Use `argmax(dim=1)` to obtain the predicted digit, then calculate accuracy as the proportion of predictions equal to the true test labels.

Fill these two dictionaries, each with keys `"linear10"` and `"mlp10"`:

- **`mnist_multiclass_test_results[name]`**: a dictionary with keys `"ce"` and `"accuracy"`, each containing a Python float.
- **`mnist_test_predictions[name]`**: a NumPy array of shape `(2000,)` containing the predicted integer digit labels from 0 to 9.

Keep `mnist_selected_multiclass` fixed at the validation-selected model; do not change it using these test results.

In [37]:

```
mnist_multiclass_test_results, mnist_test_predictions = {}, {}
for name in ("linear10", "mlp10"):
    # TODO 24: evaluate the restored model and store its metrics and predictions.
    model, criterion = mnist_models[name], mnist_criteria[name]
    model.eval()
    with torch.no_grad():
        logits = model(mnist_X_test)
        test_ce = float(criterion(logits, mnist_y_test))
        predicted_digits = logits.argmax(dim=1).numpy()
    test_accuracy = float(np.mean(predicted_digits == mnist_y_test.numpy()))
    mnist_multiclass_test_results[name] = {"ce": test_ce, "accuracy": test_accuracy}
    mnist_test_predictions[name] = predicted_digits
```

In [38]:

```
assert set(mnist_multiclass_test_results) == {"linear10", "mlp10"}
assert set(mnist_test_predictions) == {"linear10", "mlp10"}
assert mnist_selected_multiclass == mnist_selection["selected_multiclass"]
for name in ("linear10", "mlp10"):
    predictions = mnist_test_predictions[name]
    results = mnist_multiclass_test_results[name]
    assert predictions.shape == (2000,)
    assert np.issubdtype(predictions.dtype, np.integer)
    assert np.all((0 <= predictions) & (predictions <= 9))
    assert set(results) == {"ce", "accuracy"}
    assert np.isfinite(results["ce"]) and results["ce"] >= 0
    assert 0 <= results["accuracy"] <= 1
    assert np.isclose(results["accuracy"], np.mean(predictions ==
mnist_y_test.numpy()))
    print(name, results)
```

```
linear10 {'ce': 0.35594502091407776, 'accuracy': 0.904}
mlp10 {'ce': 0.2981504499912262, 'accuracy': 0.914}
```

R6 — Compare the ten-class models. Use the test CE and accuracy printed after TODO 24.

(a) Report each model's test CE and accuracy. Which model has lower CE? Which has higher accuracy? State whether the two metrics favor the same model, and report any ties. Retain the model selected on validation data.

(b) Which model can learn nonlinear scores from the pixels because it has a ReLU hidden layer? Does that model have better test CE and accuracy in this run? Explain why its more flexible structure alone does not guarantee better results on every dataset. No causal explanation of individual errors or additional learning-curve analysis is required.

Answer:

(a) `linear10`: test CE 0.3559, accuracy 0.904. `mlp10`: test CE 0.2982, accuracy 0.914. The model with lower CE is `mlp10`, and the model with higher accuracy is also `mlp10`, so the two metrics favor the same model; there are no ties. The validation-selected model `mlp10` is retained.

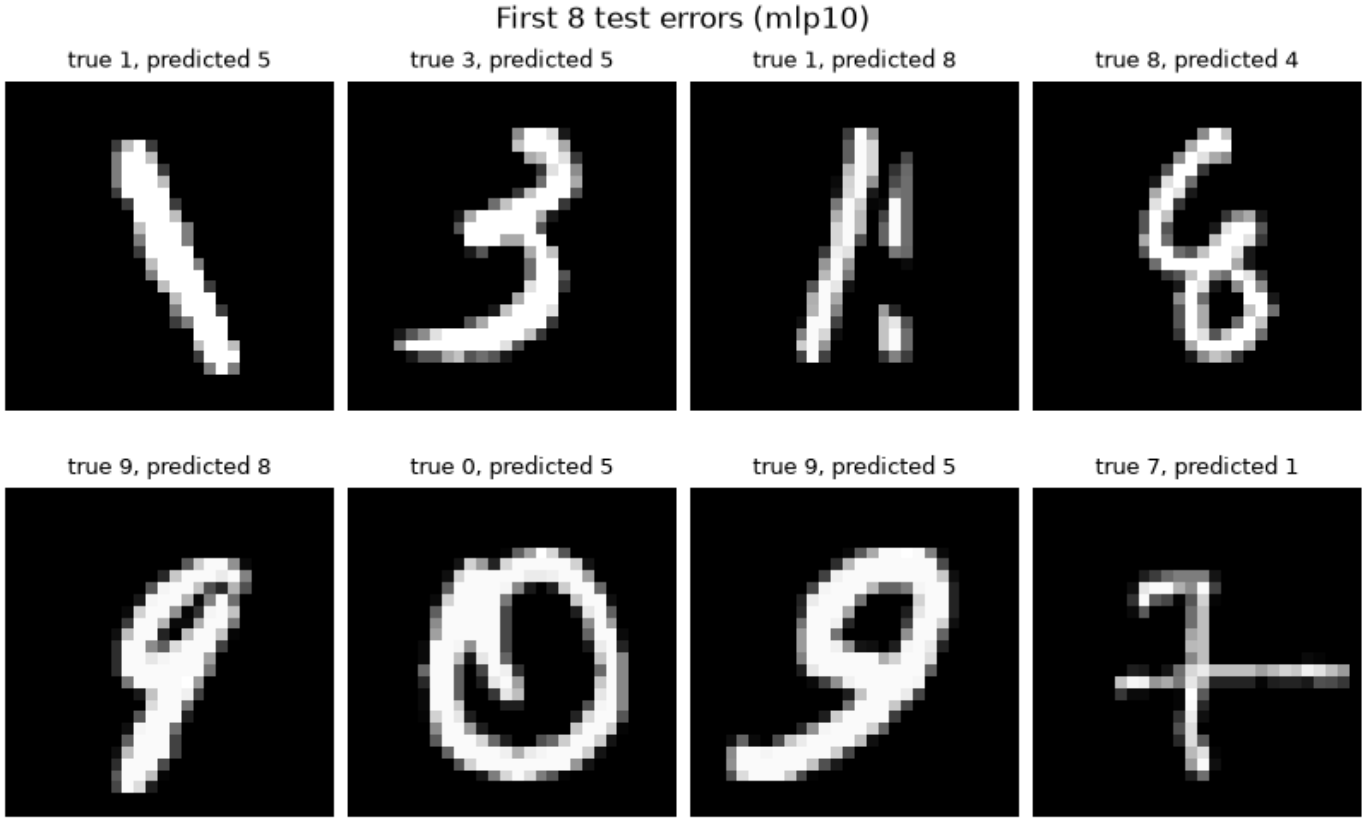
(b) `mlp10` can learn nonlinear scores from the pixels because it has a hidden layer with ReLU (784 → 64 → 10). In this run it has both better test CE (0.298 versus 0.356) and better accuracy (0.914 versus 0.904, a difference of 20 of the 2000 images). Greater flexibility alone does not guarantee better results on every dataset: a flexible model can overfit, can be harder to optimize (the `mlp10` curves show unstable spikes at the learning rate 0.1), and when a simple linear model is already adequate for the data, the extra capacity may bring little or no benefit. The outcome depends on the data, the training budget and the settings.

TODO 25: display up to eight misclassified test images from the selected ten-class model. Use the first errors in the fixed test order. Title each image with its true digit and predicted digit. Do not choose only dramatic examples. If there are no errors, print a message and do not create an empty plot. Do not use these test errors to revise the selected model.

In [39]:

```
# TODO 25: select and display a small, reproducible set of test errors.
selected_predictions = mnist_test_predictions[mnist_selected_multiclass]
error_indices = np.flatnonzero(selected_predictions != mnist_y_test.numpy())
print(f"Selected model {mnist_selected_multiclass}: {len(error_indices)} of "
      f"{len(mnist_y_test)} test images misclassified.")
if len(error_indices) == 0:
    print("No misclassified test images to display.")
else:
    shown = error_indices[:8]  # the first errors in the fixed test order
    fig, axes = plt.subplots(2, 4, figsize=(8, 5), constrained_layout=True)
    for ax in axes.ravel():
        ax.axis("off")
    for ax, index in zip(axes.ravel(), shown):
        ax.imshow(mnist_raw_test[index], cmap="gray")
        ax.set_title(f"true {int(mnist_y_test[index])}, predicted "
                     f"{int(selected_predictions[index])}", fontsize=9)
    fig.suptitle(f"First {len(shown)} test errors ({mnist_selected_multiclass})")
    plt.show()
```

```
Selected model mlp10: 172 of 2000 test images misclassified.
```



In [40]:

```
print("Saved outputs:", OUTPUT_DIR.resolve())
print("Complete S1–S8 and R1–R6; export HTML, run prepare_pdf.py, then print
executed.pdf on A4 pages.")
```

```
Saved outputs: /Users/perislyimo/Desktop/DASC
3230/PROJECTS/individual_project_2/outputs
Complete S1–S8 and R1–R6; export HTML, run prepare_pdf.py, then print executed.pdf on
A4 pages.
```

Submission

Submit **one file only: executed.pdf**. Every page must use **A4 paper (210 × 297 mm)**. The PDF must contain the complete notebook in its original order, including all code cells, execution outputs, figures, and your answers to S1–S8 and R1–R6.

1. Complete all 25 TODOs and written answers. Restart the kernel, run all cells in order, and save the notebook locally with its outputs.

2. Export the completed notebook to HTML with code and outputs visible. Save this export as **individual_project_2.html**; the next step creates **executed.html** as a separate file.

3. From a terminal in the supplied student folder, run the following command, replacing the quoted path with your exported HTML file's path:

   ```
   python3 prepare_pdf.py "/path/to/individual_project_2.html"
   ```
   The supplied tool creates **executed.html** beside your exported HTML. It adds A4 print settings, wraps long code and output lines, and fits figures to the page width. It uses only the Python standard library and is not an assessed TODO.

4. Open **executed.html** in a browser and choose **Print / Save as PDF**. Set paper size to **A4**, scale to **100%**, and margins to **Default**. Turn browser headers and footers off, then save as **executed.pdf**.

5. Open the saved PDF and inspect every page. Check that code, printed results, figures, and answers are readable and complete, with no hidden cells, clipped lines, or missing plots. Adjust the print settings if necessary.

6. Submit **executed.pdf**. Keep your `.ipynb` working file locally; do not submit the notebook, data files, model files, or a ZIP archive.

Attribution. GeoMet: Hoffimann et al. (2022), version 4, CC BY 4.0, released with obfuscated measurements/coordinates. MNIST: Yann LeCun and Corinna Cortes, MNIST, CC BY-SA 3.0 as documented by Keras. This course subset preserves the image and label values. See the data documentation and manifest for source hashes and the exact selection procedure.